Informe de Laboratorio 05

Título de la práctica: MatrLab 2.0: aplicación multiplataforma en Flutter con requisitos reales y proyección social

Nota

Integrante(s)	Escuela	Asignatura
Hilacondo Begazo, Andre Jimmy ahilacondo@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Plataformas Emergentes

Número de práctica	Título de la práctica	Docente(s)
05	MatrLab 2.0: aplicación multiplataforma en Flutter con requisitos reales y proyección social	MSc. Norman Patrick Harvey Arce

Año lectivo	Nro. Semestre	Fecha de presentación	Hora de presentación
2026	B	01/11/2026	

Índice

1. Introducción **6**
1.1. Propósito del laboratorio . . . 6
1.2. Trabajo realizado y su ubicación en el informe . . . 7
1.3. Alcance y límites . . . 7

2. Contexto real y proyección social **8**
2.1. El problema en la UNSA . . . 8
2.2. Cómo reuní la evidencia . . . 8
2.3. Hallazgos . . . 10
2.4. Proyección social . . . 11

3. Ingeniería de requisitos **12**
3.1. Cómo plantear los requisitos . . . 12
3.2. Interesados . . . 12
3.3. Árbol de problemas . . . 12
3.4. Requisitos funcionales . . . 14
3.5. Requisitos no funcionales . . . 15
3.6. Product Backlog . . . 16
3.7. Criterios de aceptación . . . 16
3.8. Matriz de trazabilidad . . . 18

4. Planificación con Scrum **19**
4.1. Scrum adaptado a un equipo de una persona . . . 19
4.2. Meta de producto y metas de sprint . . . 19
4.3. Sprint 1: turnos, explicación y atención sin fila . . . 20
4.4. Sprint 2: contingencia, comprobante y legibilidad . . . 20
4.5. Definición de terminado . . . 20
4.6. Eventos y retrospectiva . . . 21

5. Investigación de componentes de Flutter para el tema **22**
5.1. Componentes estudiados . . . 22
5.2. Decisiones de diseño con los componentes . . . 24
5.3. Paquetes que investigué y no usé . . . 24

6. Diseño de interfaces **24**
6.1. Qué cambia respecto al diseño del Laboratorio 4 . . . 24

6.2. Mapa de navegación . . . 25
6.3. Flujo de contingencia . . . 25
6.4. Mockups . . . 25

7. Desarrollo según los requisitos **33**
7.1. Preparación del proyecto . . . 33
7.2. Sprint 1 . . . 34
7.2.1. HU-08: turnos escalonados de matrícula . . . 34
7.2.2. HU-12: explicación del bloqueo y preguntas frecuentes . . . 35
7.2.3. HU-09: número de atención virtual . . . 37
7.3. Sprint 2 . . . 38
7.3.1. HU-11: modo contingencia . . . 38
7.3.2. HU-10: comprobante con código . . . 41
7.3.3. HU-13: legibilidad con texto grande . . . 42
7.4. Lo que no se construyó . . . 42

8. Recorrido de la aplicación **43**
8.1. Recorrido 1: matrícula en el turno propio . . . 44
8.2. Recorrido 2: antes de que inicie el turno . . . 45
8.3. Recorrido 3: atención sin fila . . . 45
8.4. Recorrido 4: el servicio cae . . . 46
8.5. Recorrido 5: administrador . . . 47
8.6. Casos especiales del flujo base y accesibilidad . . . 48

9. Pruebas y resultados **49**
9.1. Inventario de pruebas . . . 49
9.2. Resultados de la ejecución . . . 49
9.3. Criterios de aceptación frente a resultados . . . 50
9.4. Qué revisar primero si una prueba falla . . . 50

10. Conclusiones **50**

A. Cómo reproducir el proyecto y las capturas **52**

B. Registro de cambios del repositorio **53**

Índice de figuras

1. Árbol de problemas de MatrLab 2.0: siete causas (izquierda), problema central (centro) y cuatro efectos (derecha). . . . 13
2. Trazabilidad causa → requisito → pantalla. . . . 18
3. Flujo de trabajo: de la evidencia al Product Backlog, dos sprints con su revisión y la entrega. . . . 19
4. Mapa de navegación de MatrLab 2.0 (en amarillo, lo nuevo). . . . 26
5. Flujo de matrícula con el servicio disponible y en modo contingencia. . . . 27
6. Mockup: inicio de sesión con selector de ciclo. . . . 28
7. Mockup: pestaña Ayuda del estudiante. . . . 29
8. Mockup: número de atención virtual y preguntas frecuentes. . . . 30
9. Mockup: grupo fuera de turno. . . . 31
10. Mockup: pestaña Operación del administrador. . . . 32
11. Mockup: horario con una solicitud pendiente en modo contingencia. . . . 33
12. HU-08: el estudiante de ciclo 5 ve que su turno inicia en dos días y no puede matricularse todavía. . . . 36
13. HU-12: el sistema dice por qué no se puede continuar y qué hacer. . . . 37
14. HU-09: el estudiante recibe su número y puede esperar donde esté; el administrador atiende en orden. . . . 38
15. HU-11, primera parte: el servicio cae y la aplicación sigue aceptando solicitudes. . . . 40
16. HU-11, segunda parte: la solicitud queda visible y, al volver el servicio, se procesa y se avisa. . . . 40
17. HU-10: la confirmación muestra el comprobante (captura c08, regenerada al ejecutar las pruebas). . . . 42
18. HU-13 y RNF6: la misma pantalla con texto ampliado y en una tableta. . . . 43
19. Pasos 1 a 4: entrar con el ciclo, ver el aviso de turno, elegir un grupo y confirmar. . . . 44
20. Pasos 5 a 7: constancia con código copiable, horario y avisos. . . . 44
21. El estudiante de ciclo 5 ve cuándo le toca y por qué todavía no puede matricularse. 45
22. El estudiante pide su número, ve su posición y consulta las reglas. . . . 45
23. Modo contingencia: el banner avisa, la solicitud se guarda y, al restablecerse el servicio, se procesa y se informa en Alertas. . . . 46
24. El administrador revisa la ocupación, simula una caída y atiende a quienes esperan. 47
25. Casos del Laboratorio 4 que se conservan y pantalla con texto ampliado. . . . 48
26. Pestaña Ayuda en una tableta (riel lateral de navegación). . . . 48

Índice de tablas

1. Resumen de lo desarrollado y su ubicación en el informe. 7
2. Evidencias reunidas (todas consultadas el 1 de noviembre de 2026). 10
3. Hallazgos derivados de la evidencia. 11
4. Proceso de requisitos seguido en el laboratorio. 12
5. Interesados y necesidades. 12
6. Causas del árbol y su respaldo. C1 y C3 no se resuelven con una aplicación cliente (ver alcance). 13
7. Requisitos funcionales heredados del Laboratorio 4. 14
8. Requisitos funcionales nuevos de MatrLab 2.0, con atributos. 14
9. Requisitos no funcionales por característica de calidad. 15
10. Contraste del texto de las etiquetas, calculado con la fórmula de luminancia relativa de WCAG 2.2. 15
11. Product Backlog de MatrLab 2.0. Total comprometido: 26 puntos. 16
12. Criterios de aceptación por historia y prueba que los verifica. 17
13. Matriz de trazabilidad de MatrLab 2.0. 18
14. Elementos de Scrum y su adaptación al trabajo individual. 19
15. Sprint Backlog del Sprint 1 (13 puntos). 20
16. Sprint Backlog del Sprint 2 (13 puntos). 20
17. Definición de terminado (DoD). 21
18. Retrospectiva preliminar del trabajo. 22
19. Componentes de Flutter estudiados para el tema del proyecto. 24
20. Paquetes investigados y no incorporados. 24
21. Elementos de diseño agregados en MatrLab 2.0. 25
22. Archivos del Laboratorio 5. En total, `lib/` pasó de 2 863 a 3 476 líneas (contadas con `wc -l`, incluyen comentarios y líneas en blanco). 34
23. Pruebas del proyecto (88 definidas). 49
24. Estado de verificación de cada historia. 50
25. Registro de commits (`git log`), del más antiguo al más reciente, hasta el commit de diagramas y mockups. El commit que agrega este informe se registra en `historial_git.txt`. 53

1. Introducción

1.1. Propósito del laboratorio

La guía del laboratorio pide desarrollar, en dos semanas, una aplicación multiplataforma que enfrente un problema **basado en requisitos reales** y que abarque un tema de **proyección social**. La aplicación debe guardar relación con el diseño especificado en los trabajos anteriores, mostrar su desarrollo según los requerimientos planteados, basar el diseño de las interfaces en el árbol de problemas e investigar los componentes del framework relacionados con el tema del proyecto [1].

Para este trabajo continué con **MatrLab**, la aplicación de matrícula en laboratorios de la Escuela Profesional de Ingeniería de Sistemas que analicé en el Laboratorio 1, prototipé en Adalo en el Laboratorio 2 y programé en Flutter en el Laboratorio 4 [2, 3, 4]. La diferencia es de método: en el Laboratorio 4 los requisitos salían de hipótesis mías; ahora salen de **casos documentados de matrícula en la Universidad Nacional de San Agustín (UNSA) durante 2026**, se especifican con atributos verificables, se priorizan en un *Product Backlog* y se construyen en dos *sprints* de Scrum adaptado a un equipo de una persona. La versión resultante es MatrLab 2.0.

1.2. Trabajo realizado y su ubicación en el informe

Pedido de la guía	Trabajo realizado	Dónde
Problema basado en requisitos reales y de proyección social	Reuní 8 evidencias públicas (prensa local, comunicados de la UNSA, INEI) sobre las matrículas de 2026 y de 2016, y las convertí en 8 hallazgos	Secc. 2
Relación con el diseño de los trabajos anteriores	Se conserva MatrLab (RF1 a RF7 del Laboratorio 4) y se amplía su árbol de problemas de 6 a 7 causas, con trazabilidad a los requisitos nuevos	Secc. 3
Cómo plantear los requisitos	Proceso en cinco pasos, 5 requisitos funcionales nuevos con atributos, 6 no funcionales por ISO/IEC 25010, backlog con puntos y prioridad MoSCoW, criterios de aceptación Dado/Cuando/Entonces y matriz de trazabilidad	Secc. 3
Proceso Scrum	Meta de producto, dos sprints con meta y capacidad, definición de terminado, eventos adaptados y retrospectiva	Secc. 4
Investigar componentes del framework para el tema	Estudié 12 componentes de Flutter (selección, retroalimentación, accesibilidad, estado y tiempo) y descarté tres paquetes con su razón	Secc. 5
Interfaces basadas en el árbol de problemas	Dos pestañas nuevas (Ayuda y Operación), cuatro pantallas modificadas y seis mockups con cada elemento asociado a su widget	Secc. 6
Mostrar el desarrollo según los requerimientos	Cada historia de usuario se muestra con su código, su captura y su prueba	Secc. 7
Mostrar el sistema	Recorrido solo con capturas: estudiante, casos especiales, administrador y tableta (30 capturas en total)	Secc. 8
Resultados de la ejecución	Previstos para analizar el código, correr 88 pruebas y generar las capturas; quedan **pendientes** de la ejecución en mi equipo	Secc. 9

Tabla 1: Resumen de lo desarrollado y su ubicación en el informe.

1.3. Alcance y límites

- **Qué es:** una aplicación Flutter con 11 pantallas, lógica de matrícula con turnos por ciclo, número de atención virtual, comprobante con código y modo de contingencia, más un panel de operación para el administrador.
- **Qué no es:** no tiene servidor ni base de datos; los datos viven en memoria y se reinician al cerrar la aplicación. **La caída del servicio es una simulación** que activa el administrador con un interruptor; no mide ni mejora la capacidad real de un servidor. La concurrencia masiva (la causa C1 del árbol) solo se resuelve con infraestructura, y así lo reconoce el propio director de Servicios Académicos de la UNSA [11].
- **Evidencia:** es documental (noticias y comunicados públicos). **No realicé encuestas ni entrevistas** a estudiantes; por eso los datos de usuarios que aparecen son los que las fuentes citan, y lo demás son decisiones de diseño que se marcan como hipótesis.
- **Capturas:** se generan ejecutando la aplicación con el motor de pruebas de Flutter (`flutter_test`);

no son fotos de un teléfono físico. Las 13 capturas nuevas (c18 a c30) y los resultados de las pruebas se incorporan cuando ejecuto el proyecto (sección 9).

- **Datos de ejemplo:** los cursos y grupos son los del Laboratorio 4. Los turnos de matrícula, la cola de atención de demostración y los tiempos de espera son valores de demostración míos, no datos oficiales de la UNSA.

2. Contexto real y proyección social

2.1. El problema en la UNSA

La matrícula de la UNSA es virtual, obligatoria y masiva: cerca de 26 000 estudiantes en 59 programas de estudio usan el mismo sistema académico [11]. Durante 2026 la prensa local y los propios comunicados de la universidad documentaron dos episodios que afectaron a estudiantes y que encajan con el problema que venía trabajando en MatrLab desde el Laboratorio 1 [2]:

1. **Caída de los sistemas (mayo a junio).** Desde el 25 de mayo los sistemas informáticos de la universidad estuvieron fuera de servicio por un incidente eléctrico en el centro de datos. Hubo matrículas suspendidas, trámites paralizados y tesis postergadas; la universidad reconoció que no tiene infraestructura de alta disponibilidad ni servidor espejo, y el 12 de junio todavía no tenía una fecha exacta de restablecimiento total [8, 9].
2. **Matrícula del semestre B (agosto).** Con la plataforma saturada, unos 125 estudiantes pasaron la noche del 23 al 24 de agosto en carpas en el Área de Sociales para no perder cupos en cursos clave; por la mañana recibieron números de atención en un aula mientras esperaban que la plataforma se recuperara. Una estudiante se desmayó. El director de Servicios Académicos explicó que la plataforma procesa unas 4 000 matrículas por día, que sus servidores tienen varios años y que faltaban medidas como horarios escalonados y protocolos de contingencia [10, 11].

2.2. Cómo reuní la evidencia

Busqué en la web noticias y comunicados oficiales sobre matrícula, caídas del sistema y colas en la UNSA, y leí cada fuente completa antes de usarla. Retuve las ocho de la tabla 2. Las fuentes que no hablaban de matrícula (por ejemplo, notas sobre huelgas o tomas de local) las descarté. Esta técnica es *análisis documental*; tiene límites que conviene decir:

- Las noticias reportan lo que vieron los periodistas; no hay cifras propias ni muestreo de estudiantes. Por ejemplo, la nota del 24 de agosto no incluye número de afectados ni postura oficial [10].
- Las cifras de capacidad (4 000 matrículas por día) vienen de una declaración del director de Servicios Académicos, no de una medición independiente [11].
- No encontré documentos públicos que describan los turnos de matrícula por ciclo de la UNSA en 2026; la nota institucional de marzo solo fija el periodo general [6]. Los turnos que implementé son una propuesta de diseño, no una réplica de algo existente.

ID	Fuente	Fecha	Hecho relevante para MatrLab
E1	El Búho [5]	16/03/2016	Primera matrícula virtual de la UNSA para unos 27 000 estudiantes, presentada como «se acabaron las colas». El rector dijo que se habían preparado contingencias y que 47 estudiantes de Ingeniería de Sistemas apoyarían en las oficinas de las escuelas.
E2	UNSA, Imagen Institucional [6]	20/03/2026	Matrícula virtual 2026-A del 23 al 31 de marzo, obligatoria para ingresantes y regulares. La nota consultada no menciona turnos por ciclo ni por hora.
E3	UNSA, Dirección de Servicios Académicos [7]	2026 (matrícula 2026-A)	El sistema valida prerrequisitos y topes de créditos; hay que validar la matrícula e imprimir la constancia («sin la validación, la matrícula no quedará registrada»); las correcciones se piden en la escuela con ficha firmada y recibo de pago. La página no menciona cruces de horario ni cupos.
E4	Radio La Única [8]	01/06/2026	Más de una semana de caída de los sistemas: matrículas suspendidas, trámites paralizados, tesis postergadas. La UNSA admite que no tiene alta disponibilidad ni servidor espejo.
E5	Diario Correo [9]	12/06/2026	La caída empezó el 25 de mayo (incidente eléctrico en el centro de datos) y seguía sin solución; «no existe una fecha exacta para el restablecimiento total».
E6	Diario El Pueblo [10]	24/08/2026	El colapso «se repite al inicio de cada semestre». Estudiantes pasaron la noche a la intemperie para no perder cupos en asignaturas clave y encontraron grandes colas; uno dijo que aunque intenten matricularse a medianoche «se cae la página» y pidió un sistema menos ambiguo.

ID	Fuente	Fecha	Hecho relevante para MatrLab
E7	Diario El Pueblo [11]	25/08/2026	Unos 125 estudiantes acamparon la noche del 23 al 24 de agosto; recibieron números de atención en un aula; una estudiante se desmayó. Capacidad de unas 4 000 matrículas por día para unos 26 000 estudiantes; el proceso empezó el 10 de agosto y las solicitudes se acumularon al final. Servidores antiguos. El director reconoce que faltan mejor orientación sobre fechas, cursos y créditos, horarios escalonados y protocolos de contingencia.
E8	Andina, con datos del INEI [12]	27/12/2024	Tercer trimestre de 2024: 80,2 % de la población de 6 años o más usa internet; el 90 % de esas personas se conecta con celular. En el grupo de 19 a 24 años el uso es de 96,5 %; en el área rural, 56,1 %.

Tabla 2: Evidencias reunidas (todas consultadas el 1 de noviembre de 2026).

2.3. Hallazgos

De la tabla de evidencias saqué ocho hallazgos. Cada uno dice qué muestra la evidencia y qué implica para el diseño; ninguno afirma más de lo que dicen las fuentes.

ID	Hallazgo	Evidenci	Implicancia para MatrLab
H1	La demanda se concentra al final del periodo: el proceso empezó el 10 de agosto y las solicitudes se acumularon en los últimos días, con capacidad de 4 000 matrículas por día frente a 26 000 estudiantes.	E7, E6	Repartir la demanda con **turnos escalonados por ciclo** (RF8). El propio director lista los horarios escalonados como medida pendiente.
H2	Estudiantes pasan la noche fuera para no perder cupos, con efectos en la salud (desayuno omitido, un desmayo).	E6, E7	Que asegurar el cupo no dependa de estar presente de noche: turno claro y matrícula desde cualquier lugar (RF8, RF11).
H3	La atención asistida se organiza en persona: aula con números repartidos a mano.	E7	**Número de atención virtual** para esperar donde la persona esté (RF9).

ID	Hallazgo	Evidenci	Implicancia para MatrLab
H4	Una falla eléctrica dejó sin servicio durante semanas a miles de personas; no hay alta disponibilidad ni servidor espejo.	E4, E5	**Modo contingencia**: guardar la solicitud y procesarla por orden de llegada al restablecerse (RF11).
H5	Los servidores son antiguos y faltan protocolos de contingencia.	E7	Un protocolo visible en la aplicación: aviso de servicio, solicitudes pendientes, motivo de cada rechazo (RF11).
H6	Falta orientación sobre fechas, cursos y créditos; un estudiante califica el sistema de «ambiguo».	E7, E6	Decir **siempre el motivo** por el que no se puede matricular y qué hacer, y publicar preguntas frecuentes (RF12).
H7	La matrícula solo queda registrada si se valida; las correcciones requieren trámite presencial con ficha y recibo.	E3	Un **comprobante con código** único y copiable, como prueba verificable de la matrícula (RF10).
H8	Casi todas las personas jóvenes se conectan con celular, pero la conexión no es pareja (56,1 % en zonas rurales).	E8	Diseño para celular primero, legible con texto ampliado (RNF) y tolerante a fallos de servicio.

Tabla 3: Hallazgos derivados de la evidencia.

2.4. Proyección social

La proyección social de MatrLab es el **acceso equitativo a la matrícula**. Pasar la noche en un campus favorece a quien vive cerca, tiene tiempo libre y se siente seguro; perjudica a quien viene de provincia, trabaja o tiene problemas de salud. Esta conclusión es una inferencia mía, no un dato de las fuentes: lo que las fuentes muestran es el fenómeno (E6, E7: estudiantes a la intemperie, un desayuno omitido, un desmayo). Los cuatro requisitos principales buscan quitar la ventaja de «estar presente»:

- los **turnos escalonados** dan a cada ciclo una ventana propia en lugar de una carrera general;
- el **número virtual** reemplaza la fila física;
- el **modo contingencia** evita que una caída haga perder el lugar de quien ya intentó matricularse;
- la **explicación del bloqueo** reduce la dependencia de preguntar en persona.

Las personas beneficiadas son los estudiantes regulares (unos 26 000 en la UNSA [11]) y el personal de las direcciones de escuela, que atiende las colas. Según el INEI, la mayoría de la población joven usa internet desde el celular [12], lo que justifica una aplicación móvil. La alineación con los Objetivos de Desarrollo Sostenible 4 (educación de calidad) y 10 (reducción de las desigualdades) es una referencia de contexto, no una medición de impacto.

3. Ingeniería de requisitos

3.1. Cómo plantear los requisitos

Una lista numerada de funciones no es una especificación: cada requisito debe poder justificarse (¿de dónde sale?), priorizarse, verificarse y rastrearse hasta una pantalla y una prueba. Por eso trabajé en cinco pasos, tomando de la norma ISO/IEC/IEEE 29148 los atributos de un buen requisito (identificador, enunciado, prioridad, fuente, criterio de verificación) [17] y de Scrum la forma de ordenarlos y entregarlos [13].

N.°	Paso	Qué hice	Dónde
1	Elicitar	Análisis documental de ocho evidencias públicas y de mis informes previos; no hubo encuestas ni entrevistas.	Secc. 2
2	Analizar	Ordené los hallazgos en un árbol de problemas (causas, problema central, efectos) y distinguí lo evidenciado de lo hipotético.	Secc. 3.3
3	Especificar	Redacté requisitos funcionales y no funcionales con atributos, y los reescribí como historias de usuario con criterios de aceptación Dado/Cuando/Entonces.	Secc. 3.4 a 3.7
4	Priorizar	Clasifiqué con MoSCoW [15], estimé en puntos de historia (serie de Fibonacci) [14] y armé el *Product Backlog*.	Secc. 3.6
5	Verificar y rastrear	Cada criterio tiene una prueba automática y cada requisito una pantalla y una captura (matriz causa → requisito → historia → pantalla → prueba).	Secc. 3.8

Tabla 4: Proceso de requisitos seguido en el laboratorio.

3.2. Interesados

Interesado	Necesidad principal (según la evidencia)	Evidencia
Estudiante regular	Saber cuándo le toca matricularse, asegurar su cupo sin pasar la noche en el campus, entender por qué algo falla y tener prueba de que quedó matriculado.	E3, E6, E7
Administración de la escuela o de Servicios Académicos	Repartir la demanda en el tiempo, atender a quien necesita ayuda sin colas físicas y tener un protocolo cuando el servicio cae.	E4, E5, E7
Docente del curso (validador)	Que el proyecto se relacione con los laboratorios anteriores y muestre el desarrollo según requisitos.	Guía [1]

Tabla 5: Interesados y necesidades.

3.3. Árbol de problemas

El árbol del Laboratorio 4 tenía seis causas planteadas por mí a partir de la descripción de la aplicación [4]. Con la evidencia real lo revisé: las causas de aquel árbol (cupos poco visibles, cruces

de horario, lista de espera y falta de panel) se reúnen ahora en la causa C7, y las causas C1 a C6 son nuevas y salen de los hallazgos de la tabla 3. La figura 1 muestra el árbol y la tabla 6 indica qué respalda cada causa.

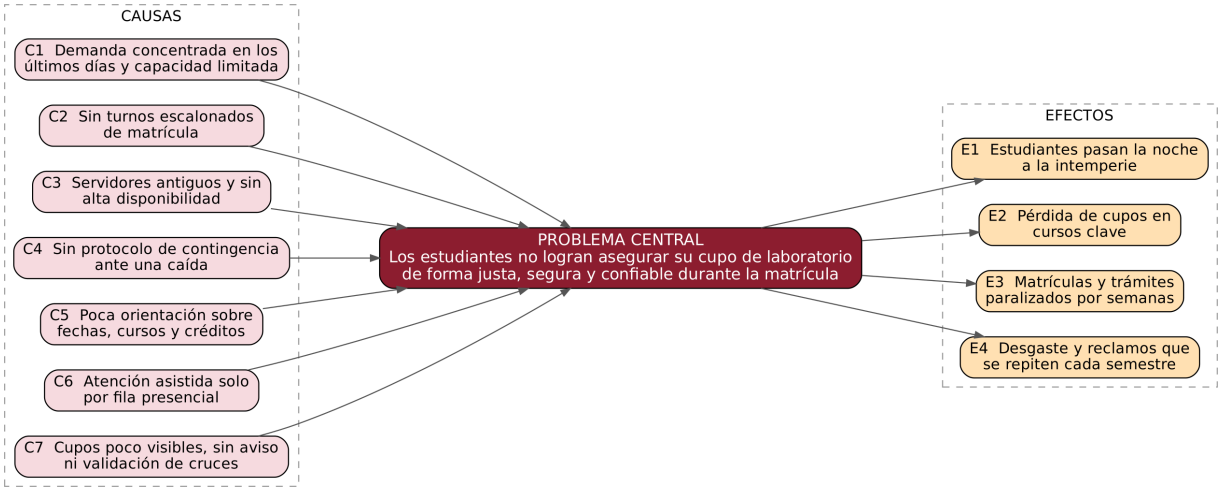


Figura 1: Árbol de problemas de MatrLab 2.0: siete causas (izquierda), problema central (centro) y cuatro efectos (derecha).

Causa	Descripción	Respaldo	Tipo
C1	Demanda concentrada en los últimos días y capacidad limitada	E7, E6	Evidenciada
C2	Sin turnos escalonados de matrícula	E7 (lo menciona como pendiente)	Evidenciada
C3	Servidores antiguos y sin alta disponibilidad	E4, E7	Evidenciada
C4	Sin protocolo de contingencia ante una caída	E4, E5, E7	Evidenciada
C5	Poca orientación sobre fechas, cursos y créditos	E6, E7	Evidenciada
C6	Atención asistida solo por fila presencial	E7	Evidenciada
C7	Cupos poco visibles, sin aviso ni validación de cruces	Laboratorios 1 y 4; E3 no menciona cruces ni cupos	Hipótesis del autor

Tabla 6: Causas del árbol y su respaldo. C1 y C3 no se resuelven con una aplicación cliente (ver alcance).

Los efectos E1 a E4 de la figura (estudiantes a la intemperie, pérdida de cupos, trámites paralizados, desgaste que se repite cada semestre) también aparecen en las fuentes (E4 a E7); su redacción es mía.

3.4. Requisitos funcionales

Los requisitos RF1 a RF7 vienen del Laboratorio 4 y se conservan sin cambio de alcance (tabla 7). Los RF8 a RF12 son nuevos y se especifican con sus atributos en la tabla 8.

ID	Requisito (Laboratorio 4)	Pantalla
RF1	Matrícula directa con descuento inmediato del cupo	Grupo, confirmación
RF2	Ocupación visible (barra, fracción y etiqueta)	Cursos, curso
RF3	Flujo mínimo curso → grupo → confirmar	Todo el flujo
RF4	Bloqueo de cruces de horario y de dos grupos del mismo curso	Grupo, horario
RF5	Lista de espera y aviso de cupo liberado	Grupo, alertas
RF6	Panel de ocupación y ajuste de cupos	Panel
RF7	Inicio de sesión con validación	Login

Tabla 7: Requisitos funcionales heredados del Laboratorio 4.

ID	Enunciado	Prioridad	Fuente	Causa	Verificación
RF8	El sistema **deberá** asignar a cada ciclo del 1 al 10 un turno de matrícula con fecha y hora de inicio y fin, mostrar al estudiante el estado de su turno (programado, en curso o finalizado) y **no deberá** permitir matricularse ni consumir cupos antes de que su turno inicie.	Must	H1, H2; E7	C1, C2	Prueba de estado y de interfaz
RF9	El sistema **deberá** permitir al estudiante pedir un número de atención virtual con un motivo, ver su posición y espera estimada, cancelarlo, y al administrador atender al siguiente; **deberá** avisar al estudiante cuando le toque. Un estudiante tendrá un solo número activo.	Must	H3; E7	C6	Prueba de estado y de interfaz
RF10	El sistema **deberá** asignar a cada matrícula confirmada un código de comprobante único y permitir copiarlo.	Should	H7; E3	C4	Prueba de estado y de interfaz
RF11	Cuando el servicio no esté disponible, el sistema **deberá** avisarlo con un banner, guardar la solicitud como pendiente (con código y sin consumir cupo), permitir cancelarla y, al restablecerse el servicio, procesar las pendientes por orden de llegada, confirmando o rechazando cada una con su motivo en Alertas.	Must	H4, H5; E4, E5	C3, C4	Prueba de estado y de interfaz
RF12	El sistema **deberá** explicar, en la pantalla del grupo, el motivo por el que no se puede matricular (turno, cruce o curso repetido) y **deberá** ofrecer preguntas frecuentes sobre las reglas.	Must	H6; E6, E7	C5	Prueba de estado y de interfaz

Tabla 8: Requisitos funcionales nuevos de MatrLab 2.0, con atributos.

3.5. Requisitos no funcionales

Los clasifiqué según las características de calidad de ISO/IEC 25010 [16] y los redacté de forma que se puedan comprobar.

ID	Característica	Requisito medible	Verificación
RNF1	Usabilidad (accesibilidad)	El estado nunca se comunica solo con color: toda etiqueta lleva icono y texto. El texto de las etiquetas tiene una relación de contraste de al menos 4,5:1 sobre su fondo (WCAG 2.2, criterio 1.4.3 [18]).	Cálculo (tabla 10)
RNF2	Usabilidad (legibilidad)	Las pantallas nuevas no se desbordan con el texto del sistema ampliado al 150 % en un teléfono de 360 dp.	Pruebas de texto grande
RNF3	Usabilidad (operabilidad)	Los botones principales miden al menos 48 dp de alto y las acciones de número y comprobante tienen etiqueta de texto.	Revisión de código
RNF4	Fiabilidad (tolerancia a fallos)	Con el servicio caído, ninguna solicitud se pierde ni consume cupo, y al restablecerse se procesan en orden de llegada.	Pruebas de HU-11
RNF5	Mantenibilidad (capacidad de prueba)	Toda regla que depende de la hora usa un reloj inyectable, de modo que las pruebas y las capturas sean deterministas; no se usan paquetes externos.	Pruebas con reloj fijo
RNF6	Portabilidad	Un solo código para Android y web, con diseño adaptable (barra inferior bajo 600 dp y riel lateral desde 600 dp).	Captura en tableta

Tabla 9: Requisitos no funcionales por característica de calidad.

Dos características de ISO/IEC 25010 quedan **fuera de alcance** a propósito: la eficiencia de desempeño bajo carga real (la capacidad de 4 000 matrículas por día depende de servidores, no de la aplicación cliente) y la seguridad (la aplicación no guarda datos reales ni autentica contra un servicio).

Comprobación del contraste (RNF1). Calculé la relación de contraste con la fórmula de WCAG para el texto de cada etiqueta sobre su fondo, que es el color del texto aplicado con 11 % de opacidad (valor 28 de 255) sobre blanco. Al revisarlo encontré que el verde que usaba desde el Laboratorio 4 (`#2E7D32`) daba 4,44:1, por debajo del mínimo de 4,5:1 para texto de este tamaño (12 dp en negrita no cuenta como texto grande). Lo corregí a `#1B5E20` en las etiquetas (`EstadoChip`, `Etiqueta` y el texto de la pestaña Ayuda). Los otros tres colores ya cumplían.

Color de etiqueta	Valor	Sobre blanco	Sobre su fondo	¿Cumple 4,5:1?
Verde (antes)	`#2E7D32`	5,13	4,44	No
Verde (ahora)	`#1B5E20`	7,87	6,61	Sí
Ámbar	`#9A5B00`	5,43	4,67	Sí
Gris (turno finalizado)	`#5F5F5F`	6,39	5,45	Sí
Rojo	`#B3261E`	6,54	5,46	Sí

Tabla 10: Contraste del texto de las etiquetas, calculado con la fórmula de luminancia relativa de WCAG 2.2.

3.6. Product Backlog

Cada requisito nuevo se redactó como historia de usuario («Como *rol*, quiero *acción* para *beneficio*») [14]. Los puntos son una estimación mía de esfuerzo relativo (serie de Fibonacci), no una medición. HU-14 y HU-15 quedan en el backlog sin construirse.

ID	Historia de usuario	MoSCoW	Pts.	Req.	Sprint
HU-08	Como estudiante, quiero ver cuándo es el turno de matrícula de mi ciclo y que el sistema respete ese turno, para no competir con todos a la vez.	Must	5	RF8	1
HU-12	Como estudiante, quiero que la aplicación me diga por qué no puedo matricularme en un grupo y qué hacer, para no depender de preguntar en persona.	Must	3	RF12	1
HU-09	Como estudiante, quiero pedir un número de atención virtual, para recibir ayuda sin hacer fila.	Must	5	RF9	1
HU-11	Como estudiante, quiero que mi solicitud se guarde si el servicio se cae, para no perder mi lugar.	Must	8	RF11, RNF4	2
HU-10	Como estudiante, quiero un comprobante con código, para demostrar que quedé matriculado.	Should	2	RF10	2
HU-13	Como estudiante con texto ampliado, quiero que las pantallas nuevas se lean sin desbordes, para usar la aplicación en mi celular.	Should	3	RNF1, RNF2	2
HU-14	Como estudiante, quiero que la aplicación recuerde mi sesión y mis solicitudes al cerrarla.	Could	5	(futuro)	Backlog
HU-15	Como estudiante, quiero recibir avisos en el celular cuando se acerque mi turno.	Won't (por ahora)	8	(futuro)	Backlog

Tabla 11: Product Backlog de MatrLab 2.0. Total comprometido: 26 puntos.

HU-14 y HU-15 requieren almacenamiento local, servidor o servicios de notificación que este laboratorio no cubre (sección 5).

3.7. Criterios de aceptación

Cada historia se considera cumplida cuando se cumplen sus criterios, escritos en la forma Dado/Cuando/Entonces. Las pruebas citadas están en la carpeta `test/` del proyecto.

HU	Criterio de aceptación	Prueba
HU-08	**Dado** un estudiante de ciclo 5 cuyo turno aún no inicia, **cuando** intenta matricularse, **entonces** el resultado es «fuera de turno», no se consume cupo y no se registra matrícula.	Estado: «fuera de turno no se consume cupo»
	Dado que el turno del estudiante inicia a una hora, **cuando** el reloj llega a esa hora, **entonces** puede matricularse.	Estado: «al iniciar el turno…»

HU	Criterio de aceptación	Prueba
	Dado el listado de turnos, **entonces** cada ciclo del 1 al 10 tiene un turno y su estado depende de la hora (programado, en curso, finalizado).	Estado: «todos los ciclos...», «el estado de cada turno...»
HU-12	**Dado** un estudiante fuera de turno, **cuando** abre un grupo, **entonces** ve el nombre de su turno, la fecha de inicio y cuánto falta, y el botón queda bloqueado con la etiqueta «Fuera de tu turno».	Estado y widget: «fuera de turno dice...», «el botón queda bloqueado»
	Dado un grupo que se cruza con otro ya matriculado, **entonces** el motivo nombra el curso en conflicto.	Estado: «el cruce de horario se explica...»
HU-09	**Dado** una cola de demostración de cuatro personas, **cuando** el estudiante pide un número, **entonces** recibe A-021, con cuatro personas delante y 20 minutos de espera estimada.	Estado: «recibe A-021...»
	Dado un estudiante con número activo, **cuando** pide otro, **entonces** no se crea. El administrador no puede pedir número.	Estado: «solo un número activo», «el administrador no puede...»
	Dado que el administrador atiende al siguiente, **cuando** el estudiante pasa a ser el primero, **entonces** recibe el aviso «Es tu turno».	Estado y widget: «al atender a los de adelante...»
HU-10	**Dado** que se confirman dos matrículas, **entonces** cada una tiene un código distinto con el prefijo MLB-2026B-, y la pantalla de confirmación permite copiarlo al portapapeles.	Estado: «código único»; widget: «se copia al portapapeles»
HU-11	**Dado** que el servicio está caído, **cuando** el estudiante confirma un grupo, **entonces** la solicitud queda pendiente con un código de prefijo SOL-2026B- y no consume cupo.	Estado: «sin servicio la solicitud queda pendiente...»
	Dado varias solicitudes pendientes, **cuando** el servicio se restablece, **entonces** se procesan por orden de llegada; la que se cruza o cuyo grupo se llenó se rechaza con su motivo en Alertas.	Estado: «se procesan por orden...», «si el grupo se llenó...»
	Dado una solicitud pendiente, **entonces** puede cancelarse y no se admiten dos para el mismo curso.	Estado: «se puede cancelar...», «no admite dos...»
HU-13	**Dado** el texto del sistema al 150 %, **cuando** se abren Ayuda, Operación y el grupo fuera de turno, **entonces** no se produce ninguna excepción de desborde.	Widget: tres pruebas de «texto grande»

Tabla 12: Criterios de aceptación por historia y prueba que los verifica.

3.8. Matriz de trazabilidad

La figura 2 resume cómo cada causa del árbol termina en un requisito y una pantalla; la tabla 13 agrega la historia, la prueba y la captura que lo documentan. Con ella se puede ir en ambos sentidos: de una causa a su evidencia y de una pantalla a su justificación.

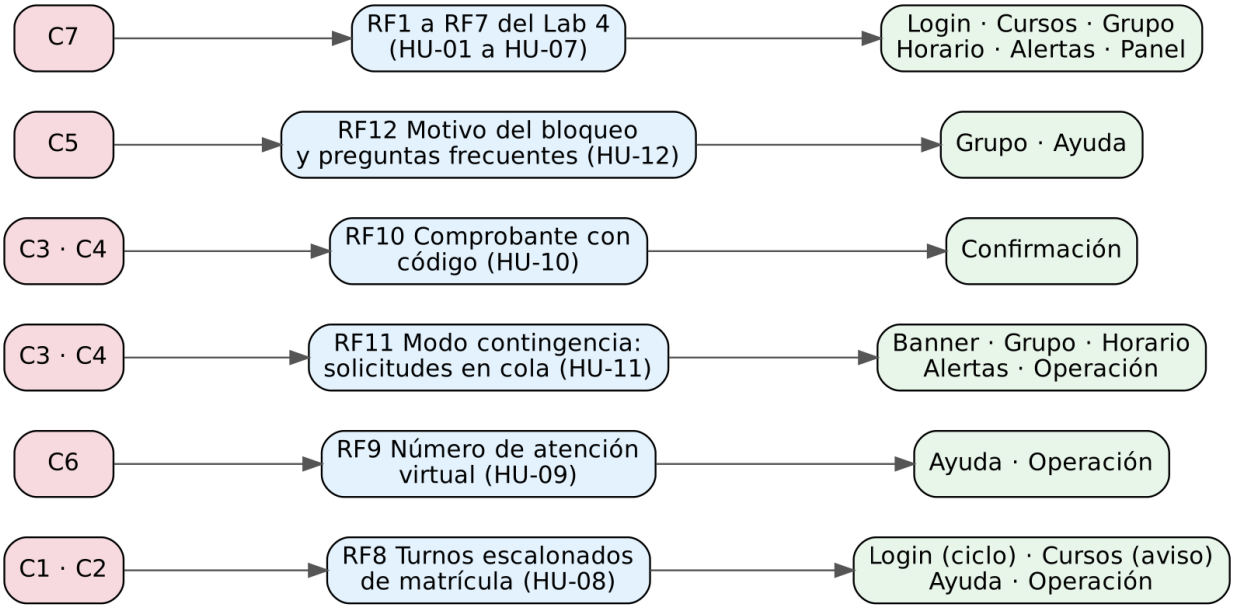


Figura 2: Trazabilidad causa → requisito → pantalla.

Causa	Req.	Historia	Pantalla	Prueba	Captura
C1, C2	RF8	HU-08	Login (ciclo), Cursos, Ayuda, Operación	`lab05_estado_test` (HU-08), `lab05_widget_test`	c18, c20
C6	RF9	HU-09	Ayuda, Operación	ídem (HU-09)	c21, c24
C4	RF10	HU-10	Confirmación	ídem (HU-10)	c08
C3, C4	RF11	HU-11	Banner, Grupo, Hoja de confirmación, Horario, Alertas, Operación	ídem (HU-11)	c23, c25 a c28
C5	RF12	HU-12	Grupo, Ayuda (preguntas)	ídem (HU-12)	c19, c22
C1 a C7	RNF2	HU-13	Ayuda, Operación, Grupo	ídem (texto grande)	c29
C7	RF1 a RF7	HU-01 a HU-07	Login, Cursos, Grupo, Horario, Alertas, Panel	`matrlab_state_test`, `flujo_widget_test`	c01 a c17

Tabla 13: Matriz de trazabilidad de MatrLab 2.0.

4. Planificación con Scrum

4.1. Scrum adaptado a un equipo de una persona

Scrum define tres responsabilidades (Product Owner, Scrum Master y Developers), tres artefactos (Product Backlog, Sprint Backlog e Incremento) y cinco eventos [13]. Este laboratorio lo hago yo solo, así que adapté el marco sin cambiar su lógica: el trabajo se ordena por valor, se entrega en iteraciones con una meta, y cada iteración termina con algo que se puede ejecutar y revisar.

Elemento de Scrum	Cómo se aplicó
Product Owner	Asumo el rol, apoyado en la evidencia de la sección 2 y en la guía del laboratorio, que cumple la función de «cliente». El docente valida en la entrega.
Developers y Scrum Master	Los asumo también. Es una limitación: no hay revisión cruzada de pares. La compensé con pruebas automáticas, trazabilidad y criterios de aceptación escritos antes de programar.
Product Backlog	Tabla 11: ocho historias con prioridad MoSCoW y puntos.
Sprint Backlog	Tareas de cada historia en las tablas 15 y 16.
Incremento	Una versión de la aplicación que se ejecuta y se prueba al cierre de cada sprint.
Sprint Planning	Elegí las historias por prioridad y dependencia; fijé una meta y una capacidad por sprint.
Daily Scrum	Se reemplaza por el registro de cambios del repositorio (anexo B): cada commit dice qué se avanzó.
Sprint Review	Revisión de capturas y de resultados de pruebas contra los criterios de aceptación (sección 9).
Sprint Retrospective	Subsección 4.6.

Tabla 14: Elementos de Scrum y su adaptación al trabajo individual.

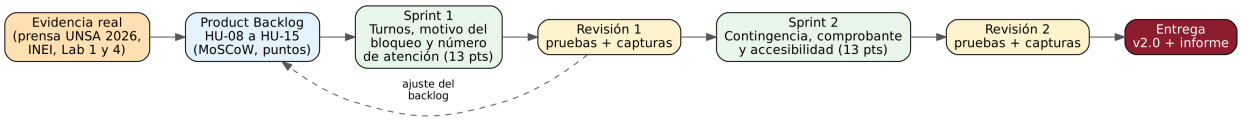


Figura 3: Flujo de trabajo: de la evidencia al Product Backlog, dos sprints con su revisión y la entrega.

4.2. Meta de producto y metas de sprint

Meta de producto. Que una persona matriculada en la UNSA pueda asegurar su cupo de laboratorio desde cualquier lugar y dentro de su turno, obtener ayuda sin hacer fila y no perder su lugar si el servicio falla.

Meta del Sprint 1. El estudiante sabe cuándo le toca matricularse, entiende por qué una acción está bloqueada y puede pedir atención sin fila (HU-08, HU-12, HU-09; 13 puntos).

Meta del Sprint 2. La matrícula no se pierde si el servicio cae, queda con un comprobante verificable y las pantallas nuevas se leen con texto ampliado (HU-11, HU-10, HU-13; 13 puntos).

Fijé una capacidad de 13 puntos por sprint como límite de trabajo en curso. Es una decisión de planificación mía; no es una velocidad medida y no la presento como tal. HU-11 (la más grande, 8 puntos) va en el Sprint 2 porque reutiliza las validaciones de turno del Sprint 1 y el comprobante de HU-10.

Nota de honestidad sobre los sprints. Los dos sprints son iteraciones lógicas, ordenadas por prioridad y dependencia; no los medí en días ni semanas, y el repositorio los registra juntos en un solo commit de código (anexo B). Por eso no presento una gráfica de avance ni una velocidad: no tengo datos para sostenerlas.

4.3. Sprint 1: turnos, explicación y atención sin fila

HU	Tareas	Archivos principales
HU-08	Modelo de turno con estados; cuatro turnos de demostración; regla que bloquea la matrícula fuera de turno; selector de ciclo en el inicio de sesión; aviso de turno en Cursos; lista de turnos en Ayuda y en Operación.	`models.dart`, `datos_demo.dart`, `estado.dart`, `login_screen.dart`, `turno_widgets.dart`
HU-12	Función que calcula el motivo del bloqueo; estados del botón del grupo; preguntas frecuentes.	`estado.dart`, `grupo_detalle_screen.dart`, `ayuda_screen.dart`
HU-09	Modelo de número de atención; solicitar, cancelar y atender; tarjeta con posición y espera; cola del administrador; avisos.	`models.dart`, `estado.dart`, `ayuda_screen.dart`, `operacion_screen.dart`

Tabla 15: Sprint Backlog del Sprint 1 (13 puntos).

4.4. Sprint 2: contingencia, comprobante y legibilidad

HU	Tareas	Archivos principales
HU-11	Interruptor de caída en Operación; banner de servicio; solicitudes pendientes con código; procesamiento por orden de llegada con motivo de rechazo; sección de pendientes en el horario.	`estado.dart`, `turno_widgets.dart`, `operacion_screen.dart`, `horario_screen.dart`, `grupo_detalle_screen.dart`
HU-10	Código de comprobante en cada matrícula; fila «Comprobante» y botón para copiarlo.	`models.dart`, `estado.dart`, `confirmacion_screen.dart`
HU-13	Etiqueta semántica del número; pruebas con texto al 150 %; corrección del contraste del verde.	`ayuda_screen.dart`, `lab05_widget_test.dart`, `comunes.dart`

Tabla 16: Sprint Backlog del Sprint 2 (13 puntos).

4.5. Definición de terminado

Una historia está terminada cuando cumple todos los puntos de la tabla 17. Indico cuáles verifiqué por revisión y cuáles quedan pendientes de la ejecución.

N.°	Condición	Estado
1	Cumple todos sus criterios de aceptación (tabla 12) con pruebas automáticas definidas.	Pruebas escritas; **resultado pendiente**
2	`flutter analyze` no reporta errores.	**Pendiente** de ejecución
3	Se ve bien a 360 dp (teléfono) y a 900 dp (tableta).	Capturas c30 y c16/c17; **pendiente** de generarlas
4	No se desborda con el texto al 150 %.	Pruebas escritas; **pendiente**
5	El texto de las etiquetas cumple contraste 4,5:1.	Cumple (tabla 10)
6	No usa paquetes externos (solo el SDK).	Cumple (`pubspec.yaml`)
7	Tiene al menos una captura en el recorrido.	Capturas definidas; **pendiente**
8	Está registrada en el repositorio con un mensaje claro.	Cumple (anexo B)

Tabla 17: Definición de terminado (DoD).

4.6. Eventos y retrospectiva

Revisión de los sprints. La revisión consiste en ejecutar `flutter test` y comparar cada captura con su criterio de aceptación. Esa ejecución se hace en mi equipo; hasta que lo haga no afirmo que los criterios se cumplen.

PENDIENTE: Revisión del Sprint 1 y del Sprint 2: indicar qué criterios pasaron y cuáles fallaron, con los resultados de la sección 9.

Retrospectiva (preliminar). Lo que sé hasta ahora, sin esperar a los resultados:

Qué funcionó	Evidencia
Partir de la evidencia	Tres de las cuatro funciones principales (turnos escalonados, protocolo de contingencia y mejor orientación) coinciden con medidas que el director de Servicios Académicos nombró como pendientes [11]; no las inventé.
Reloj inyectable	Los turnos dependen de la hora; con un reloj fijo las pruebas y las capturas son repetibles y los textos como «faltan 2 d 0 h» no cambian.
Revisar el propio diseño	Calcular el contraste de las etiquetas reveló que el verde heredado del Laboratorio 4 no cumplía; se corrigió.
Qué mejorar	**Acción**
Un commit por historia	El registro de cambios quedó con un solo commit de código para los dos sprints, por lo que no sirve como Daily Scrum. Acción: en los próximos laboratorios, un commit por historia terminada.
Verificar más temprano	Ejecutar `flutter analyze` y `flutter test` al final de cada sprint y no solo al cierre del laboratorio (el script `ejecutar.ps1` lo hace en un paso).
Datos propios de usuarios	La evidencia es documental. Con más tiempo, una encuesta breve a compañeros validaría los hallazgos H1 a H8 y ajustaría el backlog.
Trabajo en solitario	Sin revisión de pares, un error de criterio solo lo detectan las pruebas. Acción: pedir una revisión corta del informe y del código a un compañero antes de entregar.

Tabla 18: Retrospectiva preliminar del trabajo.

5. Investigación de componentes de Flutter para el tema

La guía pide investigar los componentes propios del framework *enfocados en el tema del proyecto*. En el Laboratorio 4 estudié los componentes de layout [4]; aquí estudio los que resuelven los problemas de esta versión: elegir un ciclo, explicar reglas, avisar que el servicio cayó, entregar un comprobante, hacerlo accesible y probarlo. Para cada necesidad comparo el componente elegido con su alternativa más cercana y digo por qué lo elegí.

5.1. Componentes estudiados

Necesidad	Componente elegido	Por qué, frente a la alternativa	Uso en el código
Elegir el ciclo (10 opciones)	`DropdownMenu<int>`	Es el menú de Material que se abre desde un campo de texto y muestra la opción elegida en él [21]. `SegmentedButton`, que usé para el rol, sirve para pocas opciones (2 a 5); diez no caben en un teléfono.	`login_screen.dart`, clave `menu-ciclo`
Activar la caída simulada	`SwitchListTile`	Une un interruptor con título y subtítulo en una fila completamente táctil; un `Switch` suelto exigiría armar la fila a mano.	`operacion_screen.dart`, `sw-servicio`

Necesidad	Componente elegido	Por qué, frente a la alternativa	Uso en el código
Preguntas frecuentes	`ExpansionTile`	Expande y contrae contenido con animación y ya trae el icono de expansión. Una lista de textos largos ocuparía toda la pantalla.	`ayuda_screen.dart`
Elegir el motivo de la atención	`showDialog` con `SimpleDialog`	Una lista corta de opciones excluyentes; una hoja inferior sería válida, pero el diálogo basta para cuatro opciones.	`ayuda_screen.dart` (`_pedirNumero`)
Avisar «Comprobante copiado»	`SnackBar` con `ScaffoldMessenger`	Mensaje breve que desaparece solo y no bloquea. Un diálogo interrumpiría una acción de un solo toque.	`confirmacion_screen.dart`
Copiar el código	`Clipboard.setData` (paquete `services`)	Es parte del SDK; no hace falta un paquete externo.	`confirmacion_screen.dart`, `btn-copiar-comprobante`
Etiquetas de estado	Widget propio `Etiqueta` (`Container` + `Row`)	`Chip` es el componente estándar; usé uno propio para controlar el color del texto y del fondo y poder garantizar el contraste, y para igualar a `EstadoChip` del Laboratorio 4.	`turno_widgets.dart`
Aviso de servicio caído	Banner propio (`Container` + `Row`) con `Semantics`	`MaterialBanner` se muestra mediante `ScaffoldMessenger` y está pensado para mensajes descartables con acciones. Este aviso debe permanecer mientras dure la caída y no tiene acción.	`turno_widgets.dart` (`BannerServicio`)
Lectura con lector de pantalla	`Semantics`, `ExcludeSemantics`	Permiten dar una etiqueta hablada («Tu número de atención es A-021») y evitar que se lea dos veces el texto grande. La guía de accesibilidad de Flutter pide probar con lectores de pantalla (TalkBack, VoiceOver) [19].	`ayuda_screen.dart`, `turno_widgets.dart`
Texto ampliado	Escala de texto del sistema (`TextScaler`, `MediaQuery`)	La guía de accesibilidad incluye comprobar que la interfaz se lea con factores de escala de texto muy grandes [19]; en el código esto significa no fijar alturas rígidas y comprobar que no haya desbordes.	Pruebas de HU-13
Estado y hora	`ChangeNotifier` + `InheritedNotifier`; reloj como función inyectable	`InheritedNotifier` reconstruye solo a quienes lo leen [22]. Inyectar la hora (`typedef Reloj = DateTime Function()`) permite probar turnos sin esperar.	`estado.dart`

Necesidad	Componente elegido	Por qué, frente a la alternativa	Uso en el código
Probar la interfaz	`flutter_test`: `testWidgets`, `tester.view`, canal de plataforma simulado	Permite montar la aplicación a tamaño de teléfono, ampliar el texto y simular el portapapeles sin dispositivo [23].	`lab05_widget_test.dart`

Tabla 19: Componentes de Flutter estudiados para el tema del proyecto.

5.2. Decisiones de diseño con los componentes

- **El estado nunca depende solo del color.** `Etiqueta` y `EstadoChip` combinan icono, texto y color; el botón bloqueado del grupo lleva el texto «Fuera de tu turno», no solo un tono gris. Es el criterio de WCAG 1.4.1 (uso del color) [18].
- **Un solo lugar para las reglas.** Las pantallas no deciden si se puede matricular; preguntan a `MatrLabState` (`motivoBloqueo`, `matricular`). Así el mismo motivo aparece en la pantalla del grupo y en las pruebas.
- **Las pantallas largas se desplazan.** Ayuda y Operación usan `SingleChildScrollView` dentro de `ContenidoCentrado` (ancho máximo de 720 dp), de modo que el texto ampliado desplaza el contenido en lugar de desbordarlo.
- **`DropdownMenu` no se valida.** El menú siempre tiene un ciclo seleccionado (el 10 por defecto), por lo que no necesita la validación del formulario [20].

5.3. Paquetes que investigué y no usé

Tres necesidades del backlog requieren paquetes externos o servicios que este laboratorio no cubre. No los instalé ni los probé; los registro para dejar clara la frontera del alcance.

Necesidad	Historia	Por qué queda fuera
Guardar sesión y solicitudes entre ejecuciones	HU-14	Requiere almacenamiento local persistente (por ejemplo, un paquete de preferencias). Ahora los datos viven en memoria; la prueba de contingencia necesita reiniciar el estado, no persistirlo.
Detectar la pérdida real de conexión	(mejora de HU-11)	La caída se simula con un interruptor del administrador porque lo que se demuestra es la lógica de cola, no la detección de red. Un paquete de conectividad detecta la red del dispositivo, no la caída del servidor de la universidad.
Avisos en el celular fuera de la aplicación	HU-15	Requiere un servicio de notificaciones y, en la práctica, un servidor que las envíe. Los avisos actuales viven en la pestaña Alertas.

Tabla 20: Paquetes investigados y no incorporados.

6. Diseño de interfaces

6.1. Qué cambia respecto al diseño del Laboratorio 4

La guía pide que el diseño se base en las funcionalidades del árbol de problemas y que guarde relación con los trabajos anteriores. Por eso conservo el sistema de diseño de MatrLab (Material 3, color semilla guinda `#8C1D2F`, múltiplos de 4 dp, botones principales de 48 dp) [4] y agrego solo lo que los requisitos nuevos necesitan (tabla 21).

Elemento nuevo	Decisión	Requisito
Pestaña Ayuda (estudiante)	Cuarta pestaña de la barra inferior: turnos, estado del servicio, número de atención y preguntas frecuentes en una sola pantalla con desplazamiento.	RF8, RF9, RF12
Pestaña Operación (administrador)	Tercera pestaña: interruptor de caída simulada, turnos y cola de atención.	RF8, RF9, RF11
Etiqueta de estado	Icono + texto + color. Verde = en curso o disponible; ámbar = programado o en espera; gris = finalizado; rojo = no disponible. Todas con contraste $\geq$ 4,5:1 (tabla 10).	RNF1
Aviso de turno	Franja compacta al inicio de Cursos: «Turno 1 · Ciclos 9–10: tu turno está abierto» o «inicia en 2 d 0 h».	RF8
Banner de contingencia	Franja de color de error bajo la barra superior, visible en todas las pestañas mientras el servicio esté caído.	RF11
Estados del botón del grupo	Cuatro textos según la situación: «Matricularme» o «Guardar solicitud», «Fuera de tu turno», «Solicitud pendiente de procesar» y «Ya tienes una solicitud para este curso»; los bloqueados se deshabilitan y la causa se explica en un aviso encima.	RF8, RF11, RF12
Selector de ciclo	Menú desplegable en el inicio de sesión (solo estudiantes) con el texto «Ciclo (define tu turno)».	RF8

Tabla 21: Elementos de diseño agregados en MatrLab 2.0.

6.2. Mapa de navegación

El administrador ahora ve tres pestañas (Cursos, Panel y Operación) y el estudiante cuatro (Cursos, Mi horario, Alertas y Ayuda). Los elementos nuevos están marcados en amarillo en la figura 4.

6.3. Flujo de contingencia

La pieza de diseño más importante de esta versión es qué ocurre cuando el servicio cae. La figura 5 muestra el flujo; la decisión clave es que las reglas que no dependen del servidor (turno, curso repetido, cruce de horario) se validan siempre, y solo la reserva del cupo se pospone.

6.4. Mockups

Dibujé los mockups de las pantallas nuevas o modificadas antes de programarlas, a escala de teléfono (360×740 dp). Como en el Laboratorio 4, cada número rojo identifica un elemento y la tabla contigua indica con qué widget de Flutter se construye.

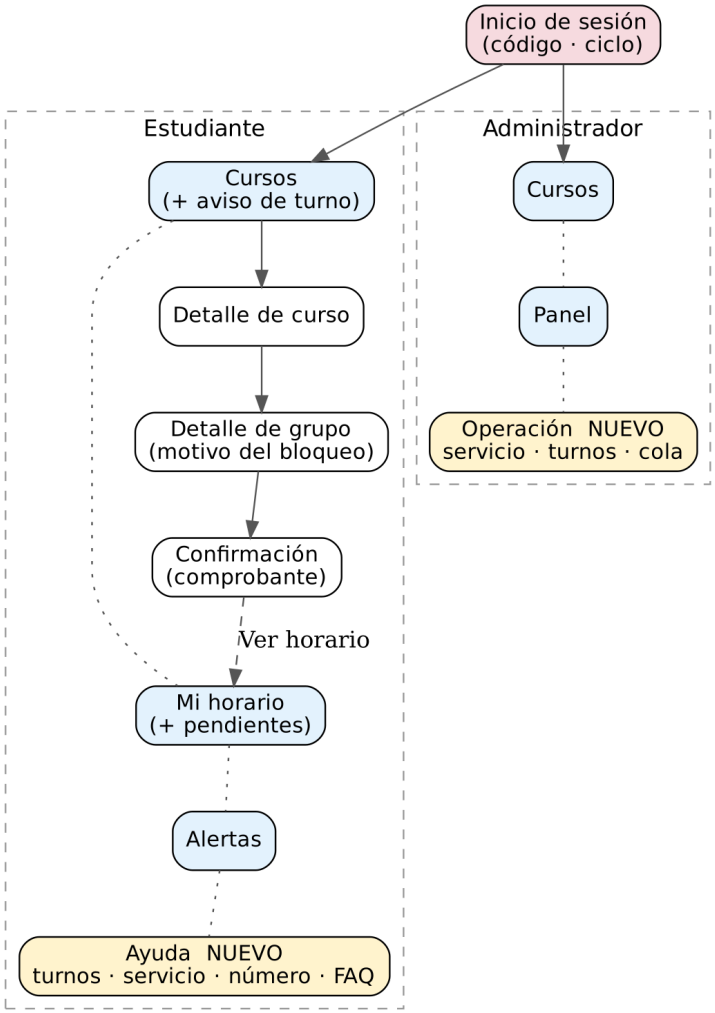


Figura 4: Mapa de navegación de MatrLab 2.0 (en amarillo, lo nuevo).

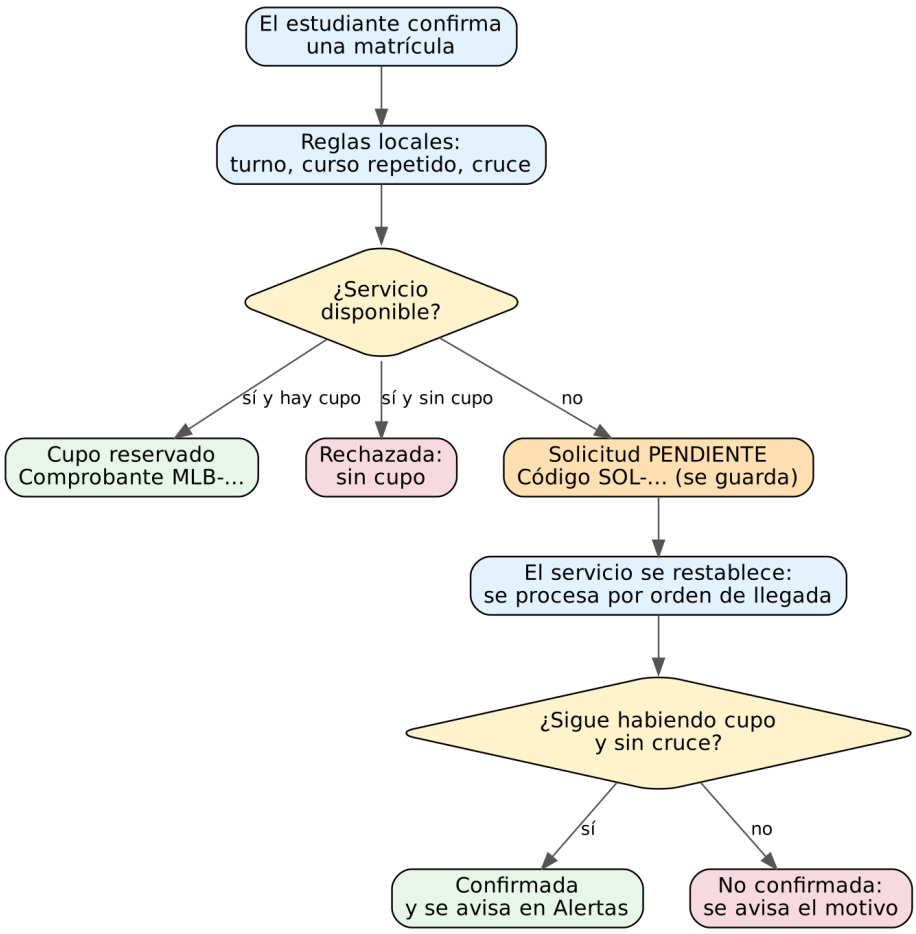


Figura 5: Flujo de matrícula con el servicio disponible y en modo contingencia.

Inicio de sesión con selección de ciclo

Cambia respecto a la versión anterior solo el campo 5: para el estudiante aparece un `DropdownMenu` que define su turno.



N.°	Elemento	Widget
1	Logo circular	`Container` circular con `Icon`
2	Selector de rol	`SegmentedButton<Rol>`
3	Código de estudiante	`TextFormField` con `validator`
4	Contraseña	`TextFormField` con `IconButton` para mostrar u ocultar
5	**Ciclo (nuevo)**	`DropdownMenu<int>` con 10 `DropdownMenuEntry`; visible solo si el rol es estudiante
6	Botón Ingresar	`FilledButton`

Figura 6: Mockup: inicio de sesión con selector de ciclo.

Pestaña Ayuda

Estructura: `ContenidoCentrado` → `SingleChildScrollView` → `Column` con cuatro bloques (turnos, servicio, atención sin fila, preguntas frecuentes). El mockup muestra los tres primeros.



N.°	Elemento	Widget
1	Lista de turnos; el del estudiante resaltado	`Container` con borde (color primario si es «tu turno») → `Row`[`Text`, `Etiqueta`] y fechas en `Text`
2	Estado del servicio	`Card` → `Row`[`Icon`, `Column`] con `Etiqueta` «Disponible» o «No disponible»
3	Pedir número de atención	`FilledButton.icon` de 48 dp dentro de un `Card`; abre un `SimpleDialog` con los motivos

Figura 7: Mockup: pestaña Ayuda del estudiante.

Número de atención y preguntas frecuentes

Cuando el estudiante ya tiene número, la tarjeta del paso 3 de la figura anterior se reemplaza por esta.

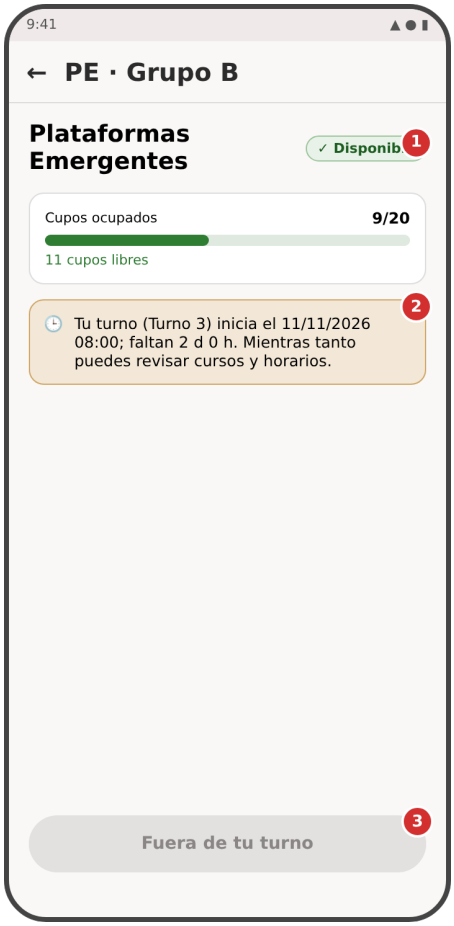


N.°	Elemento	Widget
1	Número grande (A-021)	`Text` con estilo `displaySmall` en negrita, dentro de `Semantics` + `ExcludeSemantics` para que el lector diga «Tu número de atención es A-021»
2	Posición y espera	Dos `StatTile` en una `Row` de `Expanded`; nota con `bodySmall` que declara que son 5 min por persona de demostración
3	Cancelar mi número	`TextButton.icon`
4	Preguntas frecuentes	`Card` (con `clipBehavior`) → `ExpansionTile`; la primera pregunta inicia abierta

Figura 8: Mockup: número de atención virtual y preguntas frecuentes.

Detalle de grupo fuera de turno

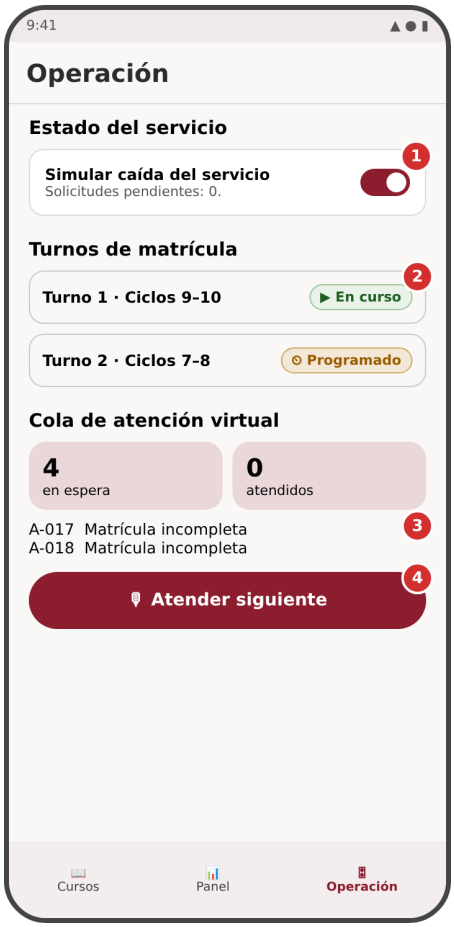
Es la respuesta de diseño a la queja de que el sistema es «ambiguo» (E6): en lugar de un botón que no hace nada, se dice el motivo y se deshabilita.



N.°	Elemento	Widget
1	Estado de ocupación	`EstadoChip` (icono + texto + color), con verde de contraste corregido
2	Motivo del bloqueo	`Aviso` (icono + `Text`) cuyo texto sale de `motivoBloqueo()`: nombre del turno, fecha y tiempo que falta
3	Botón bloqueado	`FilledButton` con `onPressed: null` y texto «Fuera de tu turno»

Figura 9: Mockup: grupo fuera de turno.

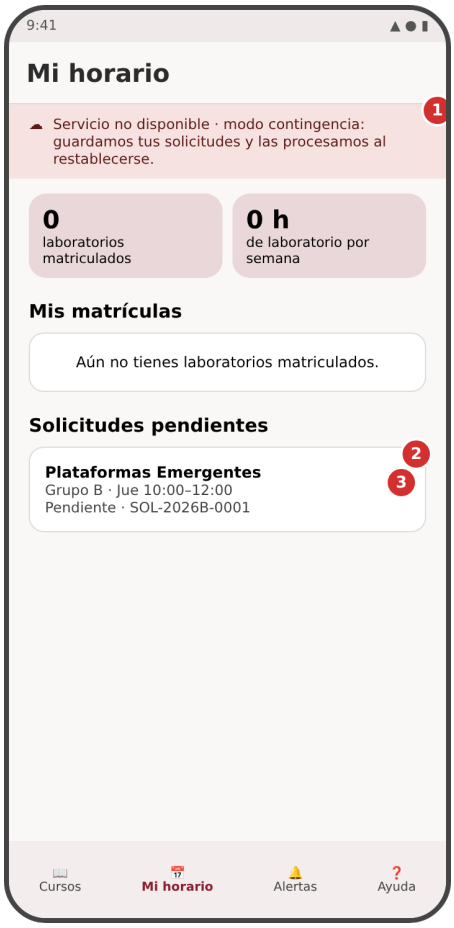
Pestaña Operación del administrador



N.°	Elemento	Widget
1	Simular caída del servicio	`Card` → `SwitchListTile`; el subtítulo muestra las solicitudes pendientes
2	Turnos de matrícula	Una fila por turno: `Container` con `Etiqueta` de estado
3	Cola de atención	Dos `StatTile` (en espera y atendidos) y filas con `SizedBox(width: 64)` para el número, `Expanded` para el motivo y la hora
4	Atender siguiente	`FilledButton.icon` de 48 dp, deshabilitado si la cola está vacía

Figura 10: Mockup: pestaña Operación del administrador.

Mi horario con solicitud pendiente



N.°	Elemento	Widget
1	Banner de contingencia	`Container` de ancho completo con color `errorContainer` y `Semantics(container: true)`, colocado sobre las pestañas en `ShellScreen`
2	Solicitud pendiente	`Card` → `ListTile` con `CircleAvatar` (icono de reloj de arena), título del curso y subtítulo con grupo, horario y código
3	Cancelar solicitud	`IconButton` con `tooltip` «Cancelar solicitud»

Figura 11: Mockup: horario con una solicitud pendiente en modo contingencia.

7. Desarrollo según los requisitos

Esta sección muestra el desarrollo ordenado por sprint e historia de usuario. Cada historia sigue el mismo orden: requisito que atiende, cómo se resolvió en Flutter, el código clave, la captura de la pantalla y la prueba que verifica el criterio de aceptación (tabla 12).

7.1. Preparación del proyecto

El proyecto de Flutter es el del Laboratorio 4, con el mismo comando de creación y sin paquetes externos. La versión pasa a 2.0.0+2 en `pubspec.yaml`. Los comandos para analizar, probar y generar las capturas son:

```
flutter pub get
flutter analyze          # analisis estatico
flutter test             # pruebas + 30 capturas en .\capturas
.\ejecutar.ps1           # lo anterior, mas salida_analyze.txt,
                         # salida_test.txt e historial_git.txt
```

Listing 1: Comandos para verificar el proyecto y generar las capturas.

Archivo	Líneas	Contenido
`lib/models.dart`	207 (modificado)	`TurnoMatricula`, `TicketAtencion`, `SolicitudPendiente`, `EstadoTicket`, código de comprobante, `formatoFaltante`
`lib/datos_demo.dart`	216 (modificado)	Cuatro turnos, cola de demostración (A-017 a A-020) y motivos de atención
`lib/estado.dart`	524 (modificado)	Reloj inyectable, turnos, motivo del bloqueo, contingencia y número de atención
`lib/widgets/turno_widgets.dart`	127 (nuevo)	`Etiqueta`, `AvisoTurno`, `BannerServicio`
`lib/screens/ayuda_screen.dart`	326 (nuevo)	Pestaña Ayuda del estudiante
`lib/screens/operacion_screen.dart`	169 (nuevo)	Pestaña Operación del administrador
`test/lab05_estado_test.dart`	254 (nuevo)	26 pruebas de reglas
`test/lab05_widget_test.dart`	265 (nuevo)	13 pruebas de interfaz
`test/capturas_test.dart`	310 (modificado)	30 capturas (17 anteriores y 13 nuevas) con reloj fijo

Tabla 22: Archivos del Laboratorio 5. En total, `lib/` pasó de 2 863 a 3 476 líneas (contadas con `wc -l`, incluyen comentarios y líneas en blanco).

7.2. Sprint 1

7.2.1. HU-08: turnos escalonados de matrícula

Requisito: RF8. **Idea:** un turno es una ventana de fechas asociada a un rango de ciclos; el estudiante solo puede matricularse cuando el reloj pasó el inicio de su turno. El turno no es un campo suelto sino un tipo con comportamiento, lo que permite que la pantalla pregunte «¿en curso?» sin repetir la comparación.

```dart
/// Ventana de matrícula asignada a un rango de ciclos (HU-08).
///
/// Un estudiante puede matricularse desde que inicia su turno y hasta que
/// termina el proceso; el escalonamiento reparte el momento de inicio, de modo
/// que no todos compiten por los mismos cupos en el mismo segundo.
class TurnoMatricula {
  const TurnoMatricula({
    required this.nombre,
    required this.cicloMin,
    required this.cicloMax,
    required this.inicio,
    required this.fin,
  });

  final String nombre;
  final int cicloMin;
  final int cicloMax;
  final DateTime inicio;
  final DateTime fin;

  bool incluyeCiclo(int ciclo) => ciclo >= cicloMin && ciclo <= cicloMax;

  bool haIniciado(DateTime ahora) => !ahora.isBefore(inicio);

  bool enCurso(DateTime ahora) => haIniciado(ahora) && ahora.isBefore(fin);

  String get rangoCiclos =>
      cicloMin == cicloMax ? 'Ciclo $cicloMin' : 'Ciclos $cicloMin-$cicloMax';

  String estadoEn(DateTime ahora) {
    if (!haIniciado(ahora)) return 'Programado';
```

```
    return enCurso(ahora) ? 'En curso' : 'Finalizado';
  }
}
```

Listing 2: Modelo del turno de matrícula (`lib/models.dart`).

El estado entrega el turno que le corresponde al ciclo del estudiante y si ya está abierto. El administrador no tiene turno. La hora sale de un reloj inyectable (`ahora`), por lo que las pruebas fijan el día y la hora.

```
  /// Turno que le corresponde al ciclo del estudiante (null si no hay).
  TurnoMatricula? get turnoDelEstudiante {
    for (final t in turnos) {
      if (t.incluyeCiclo(cicloEstudiante)) return t;
    }
    return null;
  }

  /// true si el turno del estudiante ya inició (el administrador no tiene turno).
  bool get turnoAbierto {
    if (esAdmin) return true;
    final t = turnoDelEstudiante;
    return t != null && t.haIniciado(ahora);
  }
```

Listing 3: Turno del estudiante y turno abierto (`lib/estado.dart`).

En la interfaz, el inicio de sesión recoge el ciclo con un `DropdownMenu` y la lista de cursos muestra al inicio el aviso de turno.

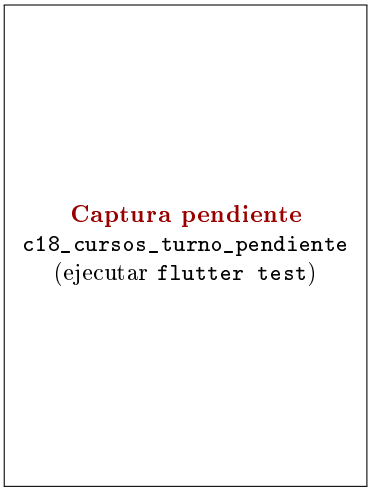
```
                    DropdownMenu<int>(
                      key: const ValueKey('menu-ciclo'),
                      expandedInsets: EdgeInsets.zero,
                      label: const Text('Ciclo (define tu turno)'),
                      leadingIcon: const Icon(Icons.event_note_outlined),
                      initialSelection: _ciclo,
                      onSelected: (v) {
                        if (v != null) setState(() => _ciclo = v);
                      },
                      dropdownMenuEntries: [
                        for (var c = 1; c <= 10; c++)
                          DropdownMenuEntry<int>(
                            value: c,
                            label: 'Ciclo $c',
                          ),
                      ],
                    ),
```

Listing 4: Selector de ciclo (`lib/screens/login_screen.dart`).

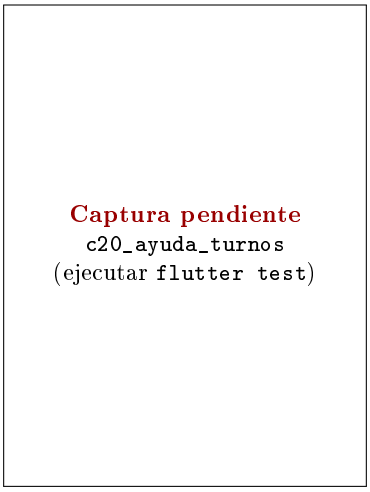
Verificación. En `lab05_estado_test.dart`: «fuera de turno no se consume cupo ni se registra matrícula», «al iniciar el turno el estudiante ya puede matricularse», «el estado de cada turno depende de la hora» y «el administrador no está sujeto a turnos». En `lab05_widget_test.dart`: «el inicio de sesión guarda el ciclo elegido».

7.2.2. HU-12: explicación del bloqueo y preguntas frecuentes

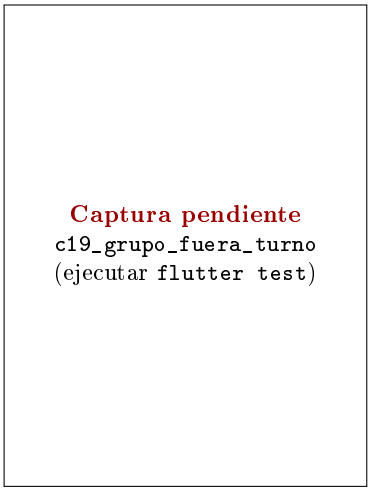
Requisito: RF12. **Idea:** la regla de por qué algo no se puede hacer vive en una sola función de `MatrLabState`, que devuelve el texto exacto para el estudiante o `null` si puede continuar. El orden importa: primero el turno, luego el curso repetido y por último el cruce de horario.



(a) Cursos con aviso de turno



(b) Ayuda: turnos



(c) Grupo fuera de turno

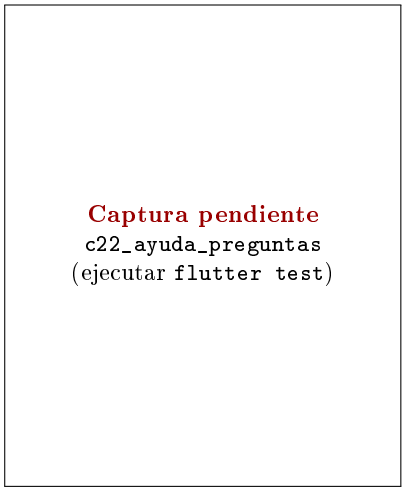
Figura 12: HU-08: el estudiante de ciclo 5 ve que su turno inicia en dos días y no puede matricularse todavía.

```dart
/// Explica por qué un estudiante no puede matricularse en [g], o null si puede
/// (HU-12: el sistema siempre dice el motivo y qué hacer).
String? motivoBloqueo(GrupoLab g) {
  if (esAdmin) return null;
  if (!turnoAbierto) {
    final t = turnoDelEstudiante;
    if (t == null) return 'Tu ciclo no tiene un turno asignado.';
    return 'Tu turno (${t.nombre}) inicia el ${formatoFechaHora(t.inicio)}; '
        'faltan ${formatoFaltante(t.inicio.difference(ahora))}. '
        'Mientras tanto puedes revisar cursos y horarios.';
  }
  final otro = grupoMatriculadoDe(g.cursoCodigo);
  if (otro != null && otro.id != g.id) {
    return 'Ya estás matriculado en ${otro.nombre} de este curso.';
  }
  final choque = conflictoCon(g);
  if (choque != null) {
    return 'Se cruza con ${cursoDeGrupo(choque).nombre} · ${choque.nombre} '
        '(${choque.horario.texto}). Elige otro grupo.';
  }
  return null;
}
```

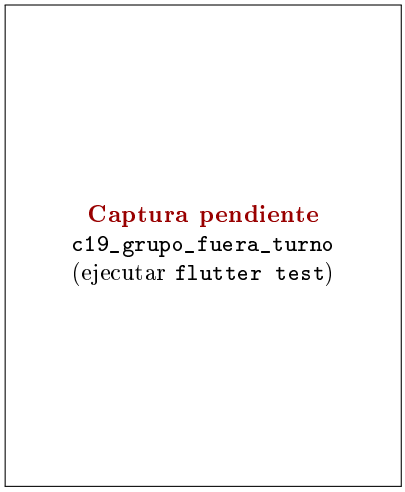
Listing 5: Motivo por el que no se puede matricular (`lib/estado.dart`).

La pantalla del grupo muestra ese texto en un aviso y deshabilita el botón. Las preguntas frecuentes de la pestaña Ayuda repiten las reglas con `ExpansionTile`.

Verificación. «fuera de turno dice cuál es el turno y cuánto falta» (el texto contiene «Turno 3» y «faltan 2 d 0 h»), «sin obstáculos no hay motivo de bloqueo», «el cruce de horario se explica con el curso en conflicto» y «un segundo grupo del mismo curso se explica». En interfaz: «fuera de turno el botón queda bloqueado y se explica».



(a) Ayuda: preguntas frecuentes



(b) Motivo en el grupo

Figura 13: HU-12: el sistema dice por qué no se puede continuar y qué hacer.

7.2.3. HU-09: número de atención virtual

Requisito: RF9. **Idea:** cada estudiante tiene como máximo un número activo; su posición se calcula contando los números en espera menores que el suyo, y la espera estimada es esa cantidad por cinco minutos (valor de demostración, declarado en pantalla). El diálogo de motivos se abre con `showDialog`.

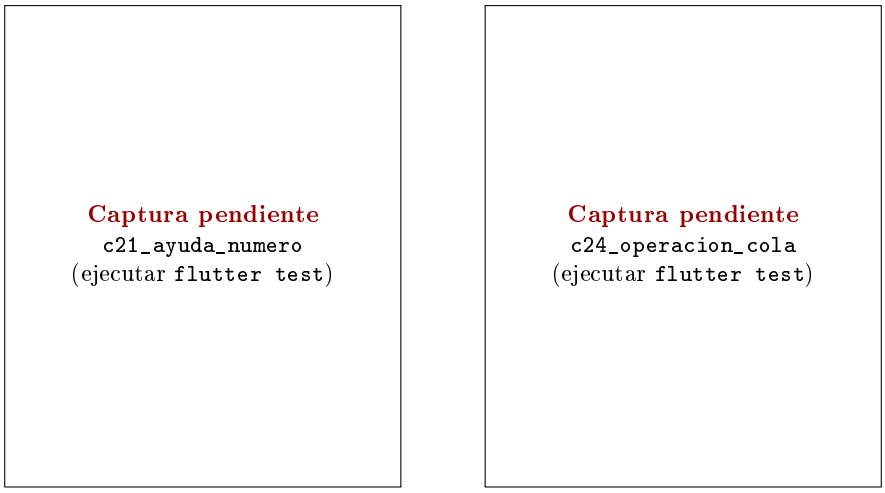
```dart
/// Pide un número de atención virtual (uno activo por estudiante).
TicketAtencion? solicitarTicket(String motivo) {
  final codigo = codigoUsuario;
  if (codigo == null || esAdmin || ticketActivo != null) return null;
  _ultimoTicket++;
  final t = TicketAtencion(
    numero: _ultimoTicket,
    codigoEstudiante: codigo,
    motivo: motivo,
    creado: ahora,
  );
  tickets.add(t);
  final delante = posicionDe(t) - 1;
  _notificar(
    'Número de atención ${t.etiqueta}',
    delante == 0
        ? 'Eres el siguiente. No necesitas hacer fila.'
        : 'Hay $delante persona(s) delante de ti. No necesitas hacer fila: '
            'te avisaremos cuando se acerque tu turno.',
    TipoNotificacion.ticket,
  );
  notifyListeners();
  return t;
}
```

Listing 6: Solicitud de un número de atención (`lib/estado.dart`).

```dart
Future<void> _pedirNumero(BuildContext context, MatrLabState e) async {
  final motivo = await showDialog<String>(
    context: context,
    builder: (ctx) => SimpleDialog(
```

```
        title: const Text('¿Por qué necesitas atención?'),
        children: [
          for (final m in motivosTicket)
            SimpleDialogOption(
              onPressed: () => Navigator.pop(ctx, m),
              child: Text(m),
            ),
        ],
      ),
    );
    if (motivo != null) e.solicitarTicket(motivo);
  }
```

Listing 7: Diálogo para elegir el motivo (`lib/screens/ayuda_screen.dart`).



(a) Número A-021 **(b)** Cola del administrador

Figura 14: HU-09: el estudiante recibe su número y puede esperar donde esté; el administrador atiende en orden.

Verificación. «la cola de demostración tiene cuatro personas», «el estudiante recibe A-021 con posición y espera estimada», «solo se permite un número activo por estudiante», «el administrador no puede pedir número», «al atender a los de adelante se avisa al estudiante», «cancelar el número permite pedir uno nuevo» y «sin cola de demostración la numeración parte de A-001». En interfaz: pedir, cancelar y atender el siguiente.

7.3. Sprint 2

7.3.1. HU-11: modo contingencia

Requisito: RF11 y RNF4. **Idea:** las reglas locales se validan siempre; si el servicio está caído, la solicitud se guarda con un código y no consume cupo. Al restablecerse, se recorre la cola en orden de llegada y cada solicitud se confirma o se rechaza con su motivo, que se comunica en Alertas.

```
  /// Reglas que no dependen del servidor (turno, curso repetido, cruce).
  ResultadoMatricula? _validarLocal(GrupoLab g) {
    if (!turnoAbierto) return ResultadoMatricula.fueraDeTurno;
    if (tieneCursoMatriculado(g.cursoCodigo)) {
      return ResultadoMatricula.yaMatriculado;
```

```
    }
    if (tienePendienteDeCurso(g.cursoCodigo)) {
      return ResultadoMatricula.yaPendiente;
    }
    if (conflictoCon(g) != null) return ResultadoMatricula.cruceHorario;
    return null;
  }
```

Listing 8: Reglas que no dependen del servidor (`lib/estado.dart`).

```
  ResultadoMatricula matricular(String grupoId) {
    final g = grupoPorId(grupoId);
    final bloqueo = _validarLocal(g);
    if (bloqueo != null) return bloqueo;

    if (!servicioDisponible) {
      // Modo contingencia: se guarda la solicitud y se decide al restablecerse.
      pendientes.add(SolicitudPendiente(
        grupoId: g.id,
        creada: ahora,
        codigo: _siguienteCodigo('SOL'),
      ));
      _notificar(
        'Solicitud guardada',
        '${cursoDeGrupo(g).nombre} · ${g.nombre}. Se procesará por orden de '
            'llegada cuando el servicio se restablezca.',
        TipoNotificacion.servicio,
      );
      notifyListeners();
      return ResultadoMatricula.pendiente;
    }

    if (g.cuposLibres <= 0) return ResultadoMatricula.sinCupo;
    _confirmar(g);
    notifyListeners();
    return ResultadoMatricula.ok;
  }
```

Listing 9: Matrícula directa o solicitud pendiente (`lib/estado.dart`).

```
  void _procesarPendientes() {
    final cola = List<SolicitudPendiente>.of(pendientes);
    pendientes.clear();
    for (final s in cola) {
      final g = grupoPorId(s.grupoId);
      final nombre = '${cursoDeGrupo(g).nombre} · ${g.nombre}';
      String? motivo;
      if (tieneCursoMatriculado(g.cursoCodigo)) {
        motivo = 'ya tenías un grupo de este curso';
      } else if (conflictoCon(g) != null) {
        motivo = 'el horario se cruza con otro grupo ya matriculado';
      } else if (g.cuposLibres <= 0) {
        motivo = 'el grupo se llenó antes de que el servicio se restableciera';
      }
      if (motivo == null) {
        _confirmar(g);
      } else {
        _notificar(
          'Solicitud no confirmada',
          '$nombre: $motivo. Elige otro grupo o únete a la lista de espera.',
          TipoNotificacion.servicio,
        );
```

```
      }
    }
  }
```

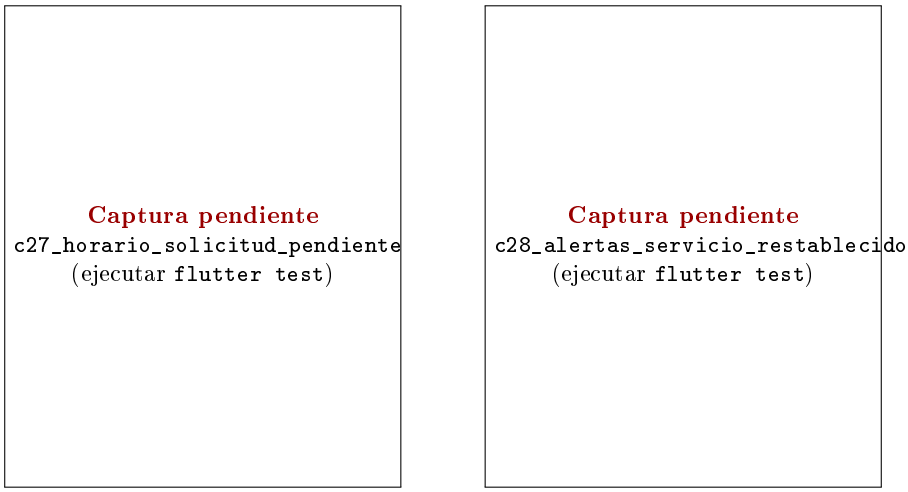
Listing 10: Procesamiento de pendientes por orden de llegada (`lib/estado.dart`).

El administrador activa la caída en la pestaña Operación; el banner aparece en todas las pestañas y los botones del grupo cambian a «Guardar solicitud».



(a) Operación: caída **(b)** Grupo con servicio caído **(c)** Guardar solicitud

Figura 15: HU-11, primera parte: el servicio cae y la aplicación sigue aceptando solicitudes.



(a) Horario con pendiente **(b)** Alertas al restablecer

Figura 16: HU-11, segunda parte: la solicitud queda visible y, al volver el servicio, se procesa y se avisa.

Verificación. «sin servicio la solicitud queda pendiente y no consume cupo», «no admite dos solicitudes para el mismo curso», «al restablecerse se confirma la solicitud y se avisa», «se procesan por orden de llegada y se rechaza la que se cruza», «si el grupo se llenó durante la caída, se rechaza con motivo», «se puede cancelar una solicitud

pendiente» y «repetir el mismo cambio de servicio no duplica avisos». En interfaz: «con el servicio caído la solicitud se guarda y luego se confirma» y «el administrador simula la caída desde el panel de operación».

7.3.2. HU-10: comprobante con código

Requisito: RF10. **Idea:** la pantalla de confirmación muestra el código de la matrícula y un botón que lo copia. `Clipboard.setData` es asíncrono; se guarda antes el `ScaffoldMessenger` para no usar el `BuildContext` después de la espera.

```
                OutlinedButton.icon(
                  key: const ValueKey('btn-copiar-comprobante'),
                  onPressed: () async {
                    final mensajero = ScaffoldMessenger.of(context);
                    await Clipboard.setData(
                        ClipboardData(text: matricula.codigo));
                    mensajero.showSnackBar(
                      const SnackBar(content: Text('Comprobante copiado')),
                    );
                  },
                  icon: const Icon(Icons.copy_outlined),
                  label: const Text('Copiar comprobante'),
                ),
```

Listing 11: Botón para copiar el comprobante (`lib/screens/confirmacion_screen.dart`).



Figura 17: HU-10: la confirmación muestra el comprobante (captura c08, regenerada al ejecutar las pruebas).

Verificación. «cada matrícula recibe un código único» y, en interfaz, «el comprobante se copia al portapapeles» (con el canal de plataforma simulado).

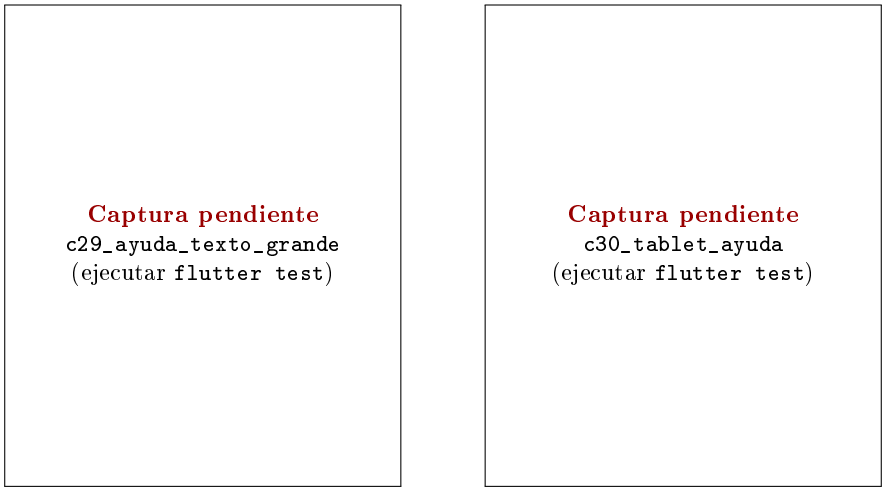
7.3.3. HU-13: legibilidad con texto grande

Requisito: RNF2 y RNF1. **Idea:** Flutter ya aplica la escala de texto del sistema; lo que se comprueba es que las pantallas nuevas no se desborden al 150 %. Las pantallas largas están dentro de un `SingleChildScrollView` y las etiquetas usan `Row` con `Expanded` para que el texto largo se ajuste. El contraste de las etiquetas se corrigió como se explicó en la sección 3.5.

Verificación. Tres pruebas de «texto grande»: Ayuda, Operación y grupo fuera de turno, cada una sin excepciones de desborde.

7.4. Lo que no se construyó

HU-14 (guardar sesión entre ejecuciones) y HU-15 (avisos en el celular) quedan en el backlog por las razones de la tabla 20. Tampoco se resuelve la capacidad del servidor (causa C1): la aplicación reparte la demanda, pero no puede aumentar el número de matrículas por día que procesa la universidad.



(a) Ayuda con texto al 150 %

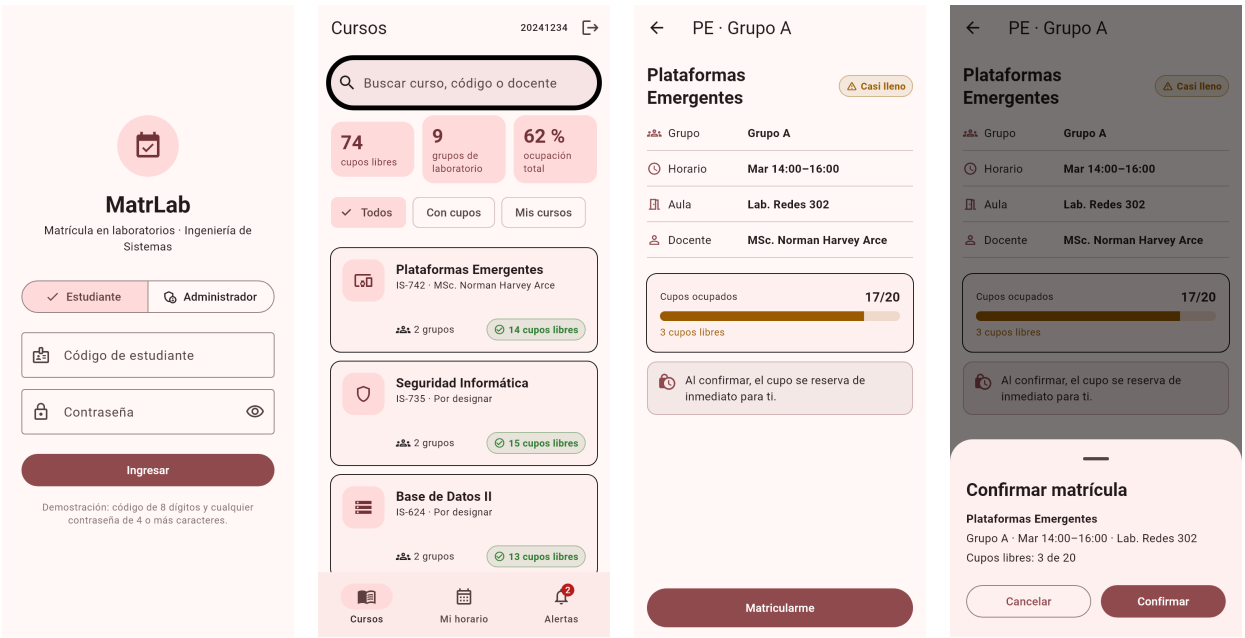
(b) Ayuda en tableta

Figura 18: HU-13 y RNF6: la misma pantalla con texto ampliado y en una tableta.

8. Recorrido de la aplicación

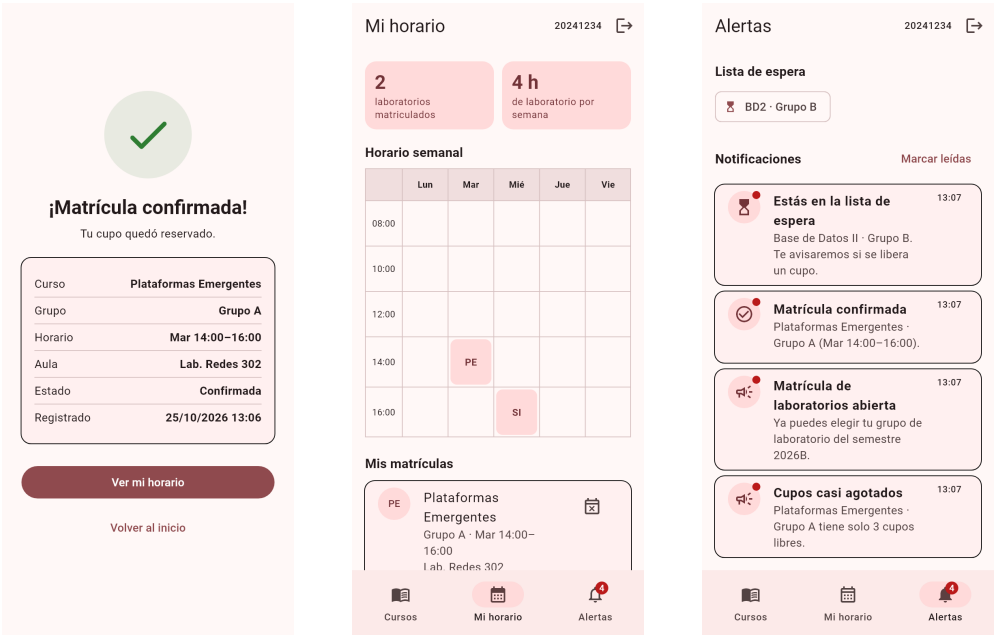
Esta sección muestra el sistema solo con capturas, en el orden en que lo recorre una persona. Son las mismas imágenes de la sección 7 más las del flujo base del Laboratorio 4, que se regeneran con el reloj fijo del 9 de noviembre de 2026 a las 08:00. El estudiante de los recorridos 1, 3 y 4 es de ciclo 10 (turno abierto) y el del recorrido 2 es de ciclo 5 (turno programado).

8.1. Recorrido 1: matrícula en el turno propio



(a) Inicio de sesión **(b)** Cursos **(c)** Grupo **(d)** Confirmar

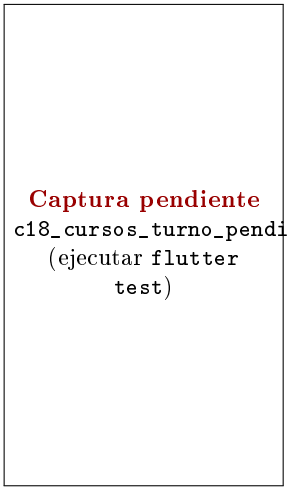
Figura 19: Pasos 1 a 4: entrar con el ciclo, ver el aviso de turno, elegir un grupo y confirmar.



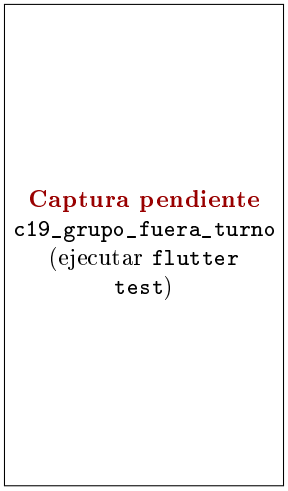
(a) Con comprobante **(b)** Mi horario **(c)** Alertas

Figura 20: Pasos 5 a 7: constancia con código copiable, horario y avisos.

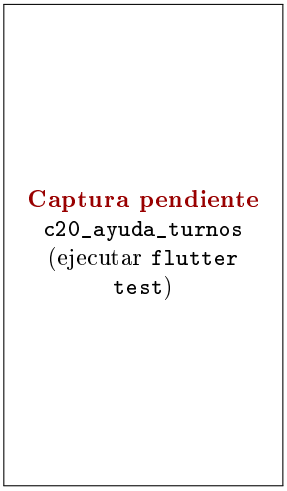
8.2. Recorrido 2: antes de que inicie el turno



(a) Aviso de turno



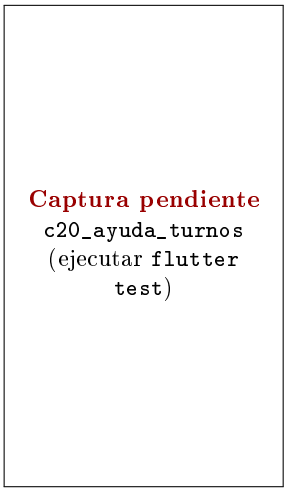
(b) Motivo y botón bloqueado



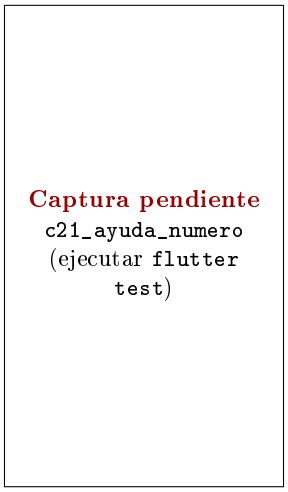
(c) Turnos en Ayuda

Figura 21: El estudiante de ciclo 5 ve cuándo le toca y por qué todavía no puede matricularse.

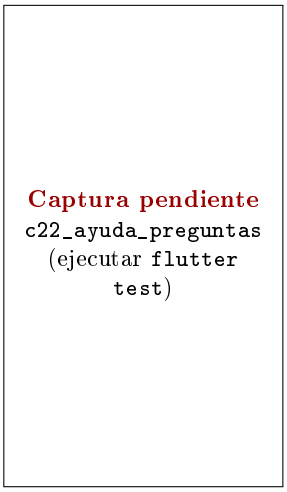
8.3. Recorrido 3: atención sin fila



(a) Ayuda



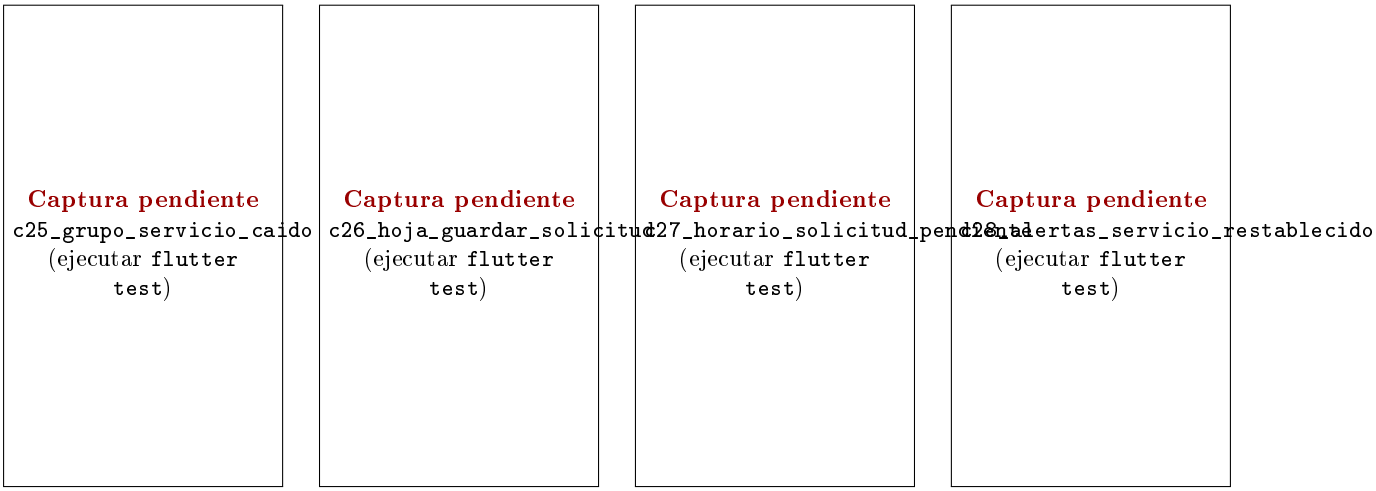
(b) Número de atención



(c) Preguntas frecuentes

Figura 22: El estudiante pide su número, ve su posición y consulta las reglas.

8.4. Recorrido 4: el servicio cae



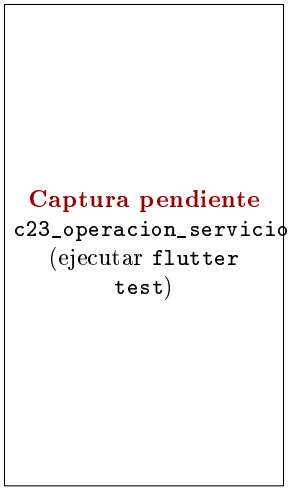
(a) Banner y grupo **(b)** Guardar solicitud **(c)** Pendiente **(d)** Servicio restablecido

Figura 23: Modo contingencia: el banner avisa, la solicitud se guarda y, al restablecerse el servicio, se procesa y se informa en Alertas.

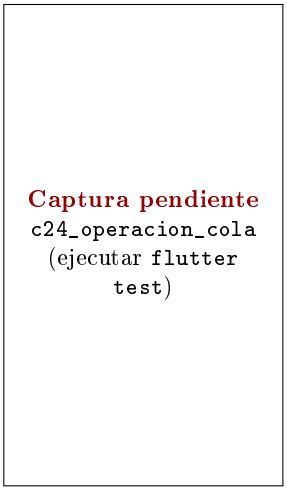
8.5. Recorrido 5: administrador



(a) Panel



(b) Operación: servicio



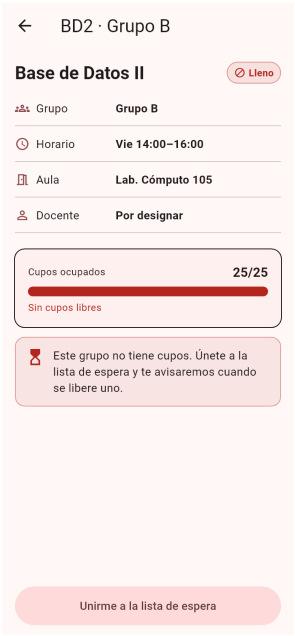
(c) Operación: cola

Figura 24: El administrador revisa la ocupación, simula una caída y atiende a quienes esperan.

8.6. Casos especiales del flujo base y accesibilidad



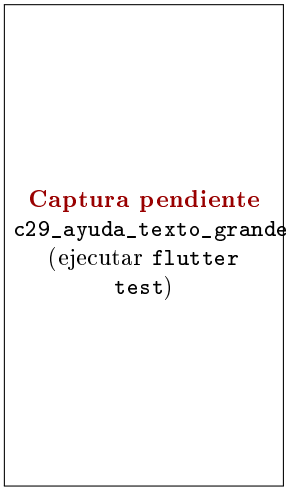
(a) Cruce de horario



(b) Grupo lleno



(c) Cupo liberado



(d) Texto al 150 %

Figura 25: Casos del Laboratorio 4 que se conservan y pantalla con texto ampliado.



Figura 26: Pestaña Ayuda en una tableta (riel lateral de navegación).

9. Pruebas y resultados

Esta sección es la que cierra el ciclo de verificación. Las pruebas están escritas y listas; los resultados dependen de ejecutarlas en mi equipo con Flutter instalado, por lo que están marcados como pendientes y **no** se afirma que pasen.

9.1. Inventario de pruebas

Archivo	Pruebas	Qué verifica	Resultado
`matrlab_state_test.dart`	17	Reglas del Laboratorio 4 (RF1 a RF7)	Pendiente
`lab05_estado_test.dart`	26	Turnos, motivo del bloqueo, comprobante, contingencia y número de atención (RF8 a RF12)	Pendiente
`flujo_widget_test.dart`	1	Flujo completo del estudiante	Pendiente
`widget_test.dart`	1	La aplicación arranca en el inicio de sesión	Pendiente
`lab05_widget_test.dart`	13	Interfaz de HU-08 a HU-13, incluido el texto al 150 %	Pendiente
`capturas_test.dart`	30	Genera las 30 capturas del informe	Pendiente
Total	**88**		

Tabla 23: Pruebas del proyecto (88 definidas).

9.2. Resultados de la ejecución

PENDIENTE: Pegar aquí la salida de `flutter analyze` (archivo `salida_analyze.txt`) y el resumen final de `flutter test` (archivo `salida_test.txt`, línea «All tests passed!» o la lista de fallos). Con ellos se completa la tabla 23 y se actualiza la definición de terminado (tabla 17).

```
$ flutter analyze
  (resultado pendiente)
$ flutter test
  (resultado pendiente: N pruebas, N correctas, N fallidas)
```

Listing 12: Formato en que se pegará la salida (pendiente).

9.3. Criterios de aceptación frente a resultados

Historia	Criterio resumido	Resultado
HU-08	Fuera de turno no se matricula ni se consume cupo; al iniciar el turno sí	Pendiente
HU-12	El motivo del bloqueo nombra turno, tiempo restante o curso en conflicto	Pendiente
HU-09	Número A-021 con posición 5 y 20 min; un solo número activo; aviso al llegar el turno	Pendiente
HU-11	Pendiente sin consumir cupo; procesamiento en orden de llegada con motivo de rechazo	Pendiente
HU-10	Código único por matrícula y copia al portapapeles	Pendiente
HU-13	Sin desbordes con texto al 150 % en Ayuda, Operación y grupo	Pendiente

Tabla 24: Estado de verificación de cada historia.

9.4. Qué revisar primero si una prueba falla

Las pruebas de interfaz dependen de detalles del motor de pruebas de Flutter que conviene mirar primero si alguna falla:

- **Selección del ciclo en el menú desplegable.** La prueba toca `menu-ciclo` y luego una opción; si el menú no se abre en el tamaño de prueba, hay que desplazar o usar la opción con `warnIfMissed: false`.
- **Escala de texto.** Las pruebas de texto grande usan `tester.platformDispatcher.textScaleFactorTestValue`; en versiones recientes del SDK puede aparecer como API en desuso. Si el analizador avisa, se cambia por la alternativa que indique el mensaje.
- **Mensajes temporales.** El `SnackBar` deja un temporizador activo; las pruebas desmontan la aplicación al final (función `_cerrar`) para evitar el error de temporizador pendiente.
- **Portapapeles.** La prueba simula el canal de plataforma `SystemChannels.platform`; si el SDK cambia el nombre del método, la prueba lo mostrará.
- **Fuentes de las capturas.** Las capturas cargan Roboto y los iconos Material desde el SDK; si faltan, el texto aparece como cuadros.

10. Conclusiones

- **Los requisitos salieron de casos reales.** Con ocho evidencias públicas sobre las matrículas de la UNSA en 2026 pasé de un árbol de problemas hipotético a uno con seis de siete causas respaldadas por fuentes. Tres de las funciones principales coinciden con medidas que la propia Dirección de Servicios Académicos reconoce como pendientes (horarios escalonados, protocolos de contingencia y mejor orientación) [11].
- **Especificar es más que enumerar.** Cada requisito tiene fuente, prioridad, criterio de aceptación, prueba, pantalla y captura. La matriz de trazabilidad permite ir de una causa a su prueba y de una pantalla a su justificación.
- **Scrum sirvió incluso en solitario**, con adaptaciones declaradas: meta de producto, backlog priorizado, dos sprints con meta, definición de terminado y retrospectiva. Su límite es la falta de revisión de pares, que compensé con pruebas automáticas.
- **El aporte social es repartir la oportunidad de matricularse**: turno propio, atención sin fila, solicitudes que no se pierden y reglas explicadas. Reduce la ventaja de quien puede estar presente de noche; no sustituye la mejora de servidores que la universidad debe hacer.

- **Un defecto propio salió al medir.** Calcular el contraste de las etiquetas mostró que el verde heredado del Laboratorio 4 no cumplía WCAG AA y se corrigió.
- **Límites reconocidos:** la evidencia es documental y no hay encuesta propia; la caída del servicio es simulada; los turnos y tiempos de espera son valores de demostración; la capacidad del servidor y las notificaciones fuera de la aplicación quedan fuera de alcance; y los resultados de las 88 pruebas están pendientes de ejecución.
- **Trabajo futuro:** validar los hallazgos con una encuesta breve a estudiantes; conectar la aplicación a un servidor para que la caída y los turnos sean reales; persistir la sesión y las solicitudes (HU-14); enviar avisos al celular (HU-15); y presentar la propuesta de turnos y número virtual a la Escuela como insumo para su proceso de matrícula.

Referencias

[1] Universidad Nacional de San Agustín de Arequipa. (2026). *Guía del Laboratorio 5: aplicación multiplataforma orientada a un problema basado en requisitos reales con proyección social.* Plataformas Emergentes, Escuela Profesional de Ingeniería de Sistemas.

[2] Hilacondo Begazo, A. J. (2026). *Informe de Laboratorio 01: comparación de herramientas de desarrollo sin código (MatrLab).* Plataformas Emergentes, UNSA.

[3] Hilacondo Begazo, A. J. (2026). *Informe de Laboratorio 02: desarrollo rápido de aplicaciones móviles: prototipo funcional en Adalo (MatrLab).* Plataformas Emergentes, UNSA, 12 de septiembre de 2026.

[4] Hilacondo Begazo, A. J. (2026). *Informe de Laboratorio 04: MatrLab: aplicación multiplataforma en Flutter para la matrícula en laboratorios.* Plataformas Emergentes, UNSA, 24 de octubre de 2026.

[5] El Búho. (16 de marzo de 2016). *Por primera vez, hoy se inicia la matrícula virtual en la Universidad San Agustín.* https://elbuho.pe/archivo/2016/03/16/por-primera-vez-hoy-se-inicia-la-matricula-virtual-en-la-universidad-san-agustin/index.html. Consultado el 1 de noviembre de 2026.

[6] Oficina Universitaria de Imagen Institucional, UNSA. (20 de marzo de 2026). *Matrículas en la UNSA inician este 23 de marzo.* https://www.unsa.edu.pe/matriculas-en-la-unsa-inician-este-23-de-marzo/. Consultado el 1 de noviembre de 2026.

[7] Dirección de Servicios Académicos, UNSA. (s. f.). *Matrícula regular (2026-A).* https://vra.unsa.edu.pe/dsa/procedimientos/matricula-regular/. Consultado el 1 de noviembre de 2026.

[8] Radio La Única. (1 de junio de 2026). *La semana en que la UNSA se quedó a oscuras.* https://radiolaunicaaqp.com/unsa-caida-sistema-problemas-gestion/. Consultado el 1 de noviembre de 2026.

[9] Diario Correo. (12 de junio de 2026). *UNSA sin sistema hace 2 semanas y afecta a miles de estudiantes de Arequipa.* https://diariocorreo.pe/edicion/arequipa/unsa-sin-sistema-hace-2-semanas-y-afecta-a-miles-de-estudiantes-de-arequipa-noticia/. Consultado el 1 de noviembre de 2026.

[10] Santander, D. N. (24 de agosto de 2026). *Sistema de la UNSA vuelven al colapso en época de matrículas.* Diario El Pueblo. https://diarioelpueblo.com.pe/2026/08/24/sistema-de-la-unsa-vuelven-al-colapso-en-epoca-de-matriculas/. Consultado el 1 de noviembre de 2026.

[11] Diario El Pueblo. (25 de agosto de 2026). *Estudiantes acampan tras caída del sistema de matrículas en la UNSA.* https://diarioelpueblo.com.pe/2026/08/25/estudiantes-acampan-tras-caida-del-sistema-de-matriculas-en-la-unsa/. Consultado el 1 de noviembre de 2026.

[12] Andina. (27 de diciembre de 2024). *INEI: uso de internet alcanzó al 80.2 % de la población de 6 y más años de edad.* https://andina.pe/agencia/noticia-inei-uso-internet-alcanzo-al-802-de-poblacion-6-y-mas-anos-edad-1012997.aspx. Consultado el 1 de noviembre de 2026.

[13] Schwaber, K., y Sutherland, J. (2020). *The Scrum Guide.* https://scrumguides.org/scrum-guide.html. Consultado el 1 de noviembre de 2026.

[14] Cohn, M. (2004). *User Stories Applied: For Agile Software Development.* Addison-Wesley.

[15] Clegg, D., y Barker, R. (1994). *Case Method Fast-Track: A RAD Approach.* Addison-Wesley.

[16] ISO/IEC. (2011). *ISO/IEC 25010:2011. Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): System and software quality models.*

[17] ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148:2018. Systems and software engineering: Life cycle processes: Requirements engineering.*

[18] World Wide Web Consortium. (2024). *Web Content Accessibility Guidelines (WCAG) 2.2.* https://www.w3.org/TR/WCAG22/. Consultado el 1 de noviembre de 2026.

[19] Flutter. (s. f.). *Accessibility.* https://docs.flutter.dev/ui/accessibility. Consultado el 1 de noviembre de 2026.

[20] Flutter. (s. f.). *Build a form with validation.* https://docs.flutter.dev/cookbook/forms/validation. Consultado el 1 de noviembre de 2026.

[21] Flutter. (s. f.). *DropdownMenu class.* API de Flutter. https://api.flutter.dev/flutter/material/DropdownMenu-class.html. Consultado el 1 de noviembre de 2026.

[22] Flutter. (s. f.). *InheritedNotifier class.* API de Flutter. https://api.flutter.dev/flutter/widgets/InheritedNotifier-class.html. Consultado el 1 de noviembre de 2026.

[23] Flutter. (s. f.). *Testing Flutter apps.* https://docs.flutter.dev/testing/overview. Consultado el 1 de noviembre de 2026.

A. Cómo reproducir el proyecto y las capturas

1. Instalar Flutter y Android Studio; comprobar con `flutter doctor`.
2. Abrir una terminal en la carpeta `matrlab-flutter` y, solo la primera vez, ejecutar `flutter create -project-name matrlab -platforms android,web .` (agrega las carpetas de plataforma sin sobrescribir `lib/` ni `pubspec.yaml`).
3. Ejecutar `.\ejecutar.ps1`, que corre `flutter pub get`, `flutter analyze` y `flutter test`; guarda las 30 capturas en `capturas/` y las salidas en `salida_analyze.txt`, `salida_test.txt` e `historial_git.txt`.
4. `flutter run` abre la aplicación en un emulador; `flutter run -d chrome`, en el navegador.
5. Credenciales de demostración: estudiante con un código de 8 dígitos (por ejemplo, `20201234`), cualquier clave de 4 caracteres o más y un ciclo (el 10 tiene el turno abierto; el 5 aún no); administrador con un código de 4 o más caracteres (por ejemplo, `admin`).
6. Para ver la contingencia: ingresar como administrador, pestaña Operación, activar «Simular caída del servicio»; luego, como estudiante, intentar matricularse.
7. Compilar este informe: `latexmk -pdf informe.tex` dentro de `informe_lab05/`.

B. Registro de cambios del repositorio

El registro de la tabla 25 se obtuvo con `git log`; las fechas son las reales de cada *commit*. Los siete primeros pertenecen al Laboratorio 4; los demás, al Laboratorio 5.

N.°	Commit	Fecha	Lab.	Mensaje
1	`0012d28`	25/10/2026 12:22	4	Lab 04 Plataformas Emergentes: MatrLab en Flutter con informe LaTeX
2	`c9281e4`	25/10/2026 12:49	4	Corrige iconos en las capturas de prueba y desborde del panel de administrador
3	`f3bffc9`	25/10/2026 12:52	4	Corrige desborde en tarjetas de curso y estabiliza las pruebas de interfaz
4	`dde5cc7`	25/10/2026 12:56	4	Ajusta la prueba de flujo: sin desplazamiento innecesario
5	`4d47b27`	25/10/2026 13:06	4	Prueba de flujo: mismo ritmo de avance que las capturas
6	`0faa6a1`	25/10/2026 13:08	4	Agrega capturas reales, resultados de las pruebas y registro de cambios 1.1
7	`da2c27c`	25/10/2026 13:11	4	Agrega la seccion de recorrido de la aplicacion solo con capturas
8	`2b1cd0b`	01/11/2026 14:07	5	Lab 05: MatrLab 2.0 con turnos, atencion virtual y modo contingencia
9	`41599fe`	01/11/2026 14:07	5	Agrega diagramas y mockups del Lab 05

Tabla 25: Registro de commits (`git log`), del más antiguo al más reciente, hasta el commit de diagramas y mockups. El commit que agrega este informe se registra en `historial_git.txt`.